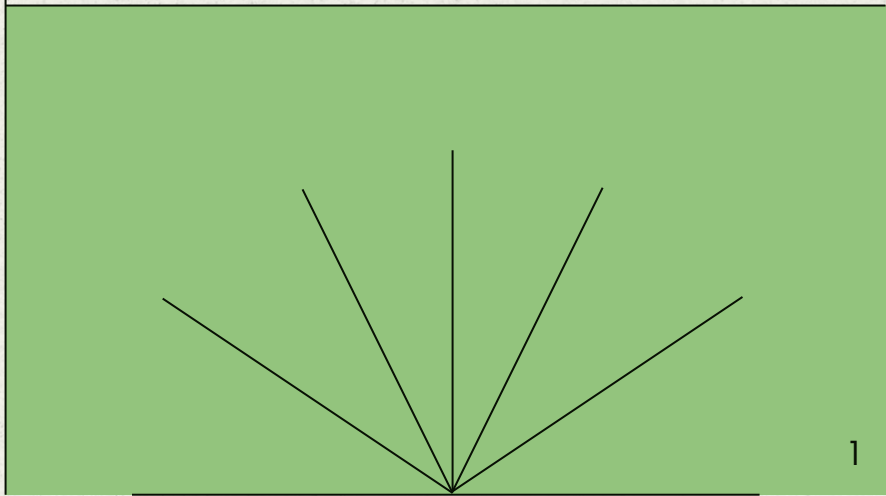


UNIVERSITEIT VAN AMSTERDAM

Lecture 6: Compositional semantics and sentence representations

Vera Neplenbroek

Credits: Sandro Pezelle, Ekaterina Shutova, J. Bastings, Mario Giulianelli, Rochelle Choenni



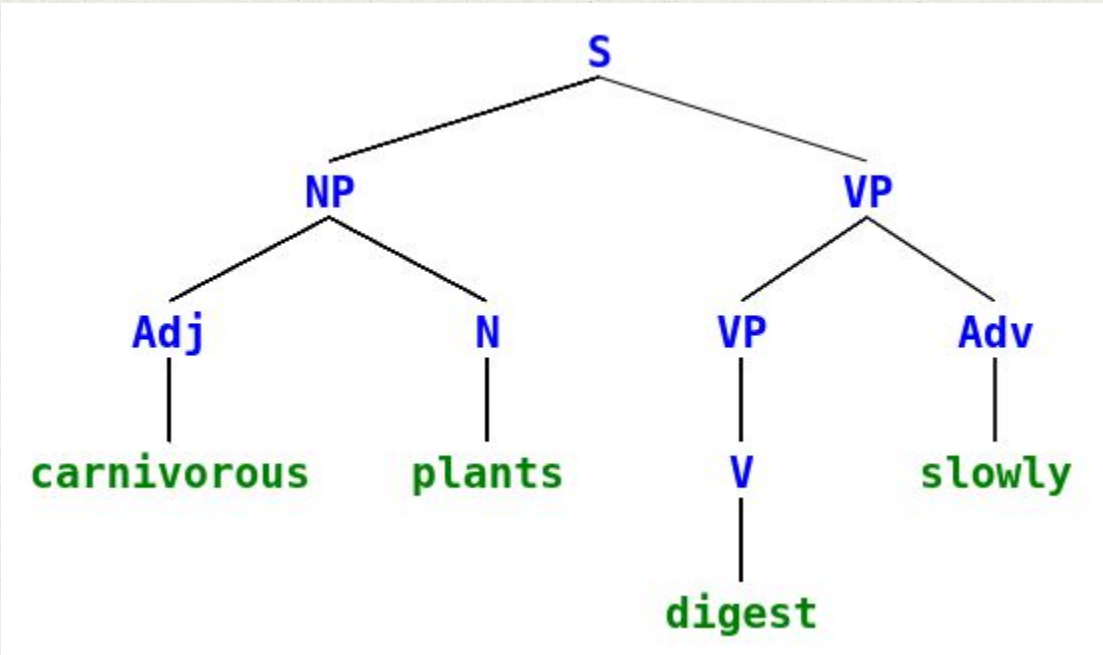
OUTLINE

- **Compositional semantics**
- Compositional distributional semantics
- Compositional semantics with neural networks

COMPOSITIONAL SEMANTICS

- **Principle of Compositionality**: meaning of each whole phrase derivable from meaning of its parts.
- Sentence structure conveys some meaning.
- **Deep grammars**: model semantics alongside syntax, one semantic composition rule per syntax rule

COMPOSITIONAL SEMANTICS



NON-TRIVIAL ISSUES WITH SEMANTIC COMPOSITION

- Similar syntactic structures may have different meanings
  - *it barks*
  - *it rains; it snows* (**pleonastic pronoun**)

NON-TRIVIAL ISSUES WITH SEMANTIC COMPOSITION

- Similar syntactic structures may have different meanings
  - *it barks*
  - *it rains; it snows* (**pleonastic pronoun**)

- Different syntactic structures may have the same meaning (e.g. passive constructions)
  - *Kim ate the apple.*
  - *The apple was eaten by Kim.*

NON-TRIVIAL ISSUES WITH SEMANTIC COMPOSITION

- Similar syntactic structures may have different meanings
  - *it barks*
  - *it rains; it snows* (**pleonastic pronoun**)

- Different syntactic structures may have the same meaning (e.g. passive constructions)
  - *Kim ate the apple.*
  - *The apple was eaten by Kim.*

- Not all phrases are interpreted compositionally (e.g., **idioms**)
  - *red tape*
  - *kick the bucket*

  but they can be interpreted compositionally too, so we cannot simply block them.

NON-TRIVIAL ISSUES WITH SEMANTIC COMPOSITION

- Additional meaning can arise through composition (e.g., **logical metonymy**)
  - *fast programmer*
  - *fast plane*
  - *enjoy a book*
  - *enjoy a cup of tea*

NON-TRIVIAL ISSUES WITH SEMANTIC COMPOSITION

- Additional meaning can arise through composition (e.g., **logical metonymy**)
  - *fast programmer*
  - *fast plane*
  - *enjoy a book*
  - *enjoy a cup of tea*

- Meaning transfers and additional connotations can arise through composition (e.g., **metaphor**)
  - *I can't **buy** this story.*
  - *This sum will **buy** you a ride on the train.*

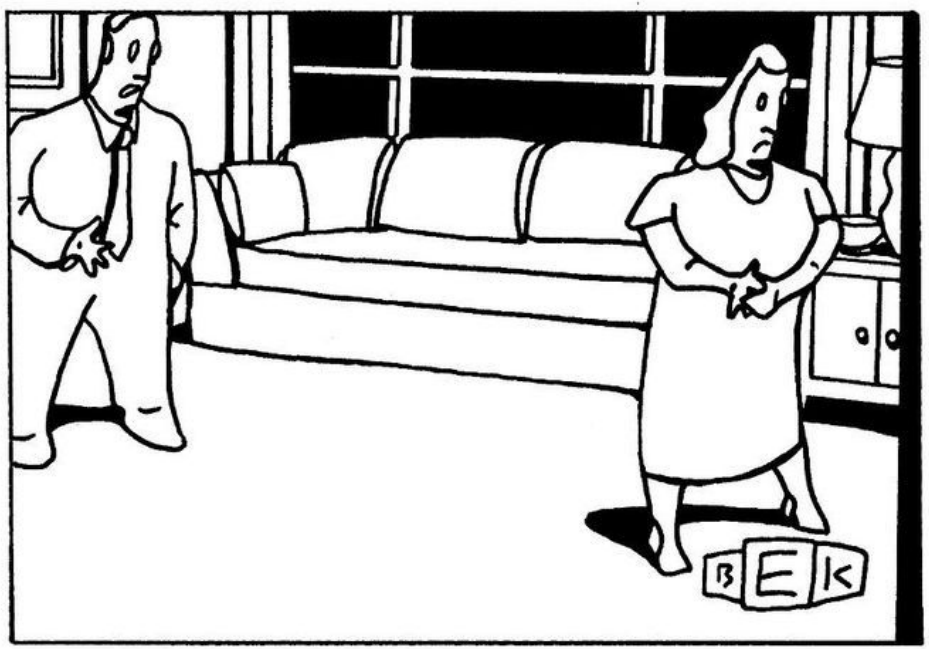
NON-TRIVIAL ISSUES WITH SEMANTIC COMPOSITION

- Additional meaning can arise through composition (e.g., **logical metonymy**)
  - *fast programmer*
  - *fast plane*
  - *enjoy a book*
  - *enjoy a cup of tea*

- Meaning transfers and additional connotations can arise through composition (e.g., **metaphor**)
  - *I can't **buy** this story.*
  - *This sum will **buy** you a ride on the train.*

- Recursive composition

NON-TRIVIAL ISSUES WITH SEMANTIC COMPOSITION



"Of course I care about how you imagined I thought you perceived I wanted you to feel."

MODELLING COMPOSITIONAL SEMANTICS

1. Compositional **distributional semantics**
   - composition is modelled in a vector space
   - unsupervised
   - general purpose representations

2. Compositional semantics with **neural networks**
   - supervised or self-supervised
   - (typically) task-specific representations

OUTLINE

- Compositional semantics
- **Compositional distributional semantics**
- Compositional semantics with neural networks

COMPOSITIONAL DISTRIBUTIONAL SEMANTICS

Can distributional semantics be extended to account for the meaning of phrases and sentences?

COMPOSITIONAL DISTRIBUTIONAL SEMANTICS

Can distributional semantics be extended to account for the meaning of phrases and sentences?

- Given a finite vocabulary, natural languages licence an infinite amount of sentences.
- So it is impossible to learn vector representations for all sentences.

COMPOSITIONAL DISTRIBUTIONAL SEMANTICS

Can distributional semantics be extended to account for the meaning of phrases and sentences?

- Given a finite vocabulary, natural languages licence an infinite amount of sentences.
- So it is impossible to learn vector representations for all sentences.

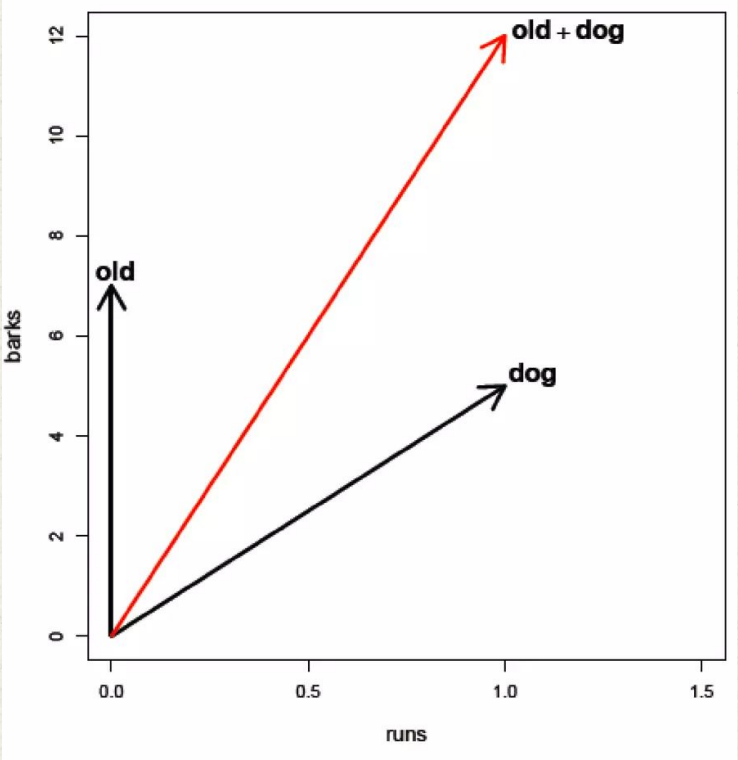
But we can still use distributional word representations and learn to perform **semantic composition in distributional space**.

VECTOR MIXTURE MODELS

Mitchell and Lapata, 2010. Composition in Distributional Models of Semantics Models

- Additive
- Multiplicative

Simple, but surprisingly effective!



ADDITIVE AND MULTIPLICATIVE MODELS

				additive		multiplicative	
	dog	cat	old	old + dog	old + cat	old ⊙ dog	old ⊙ cat
runs	1	4	0	1	4	0	0
barks	5	0	7	12	7	35	0

- Correlate with human similarity judgments about adjective-noun, noun-noun, verb-noun and noun-verb pairs

ADDITIVE AND MULTIPLICATIVE MODELS

				additive		multiplicative	
	dog	cat	old	old + dog	old + cat	old ⊙ dog	old ⊙ cat
runs	1	4	0	1	4	0	0
barks	5	0	7	12	7	35	0

- Correlate with human similarity judgments about adjective-noun, noun-noun, verb-noun and noun-verb pairs
- The additive and the multiplicative model are **symmetric** (commutative): They do not take word order or syntax into account.
  - *John hit the ball = The ball hit John*

ADDITIVE AND MULTIPLICATIVE MODELS

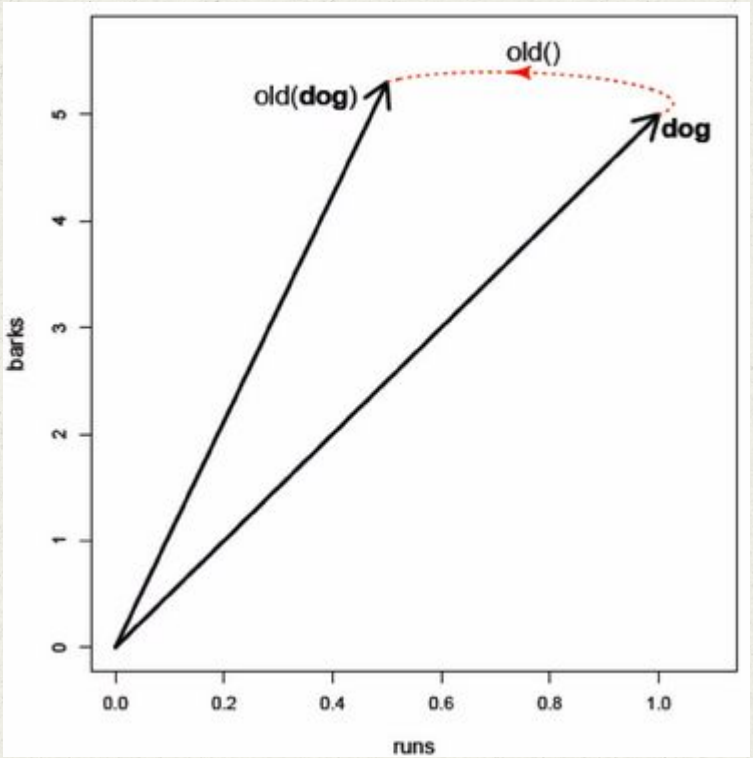
				additive		multiplicative	
	dog	cat	old	old + dog	old + cat	old ⊙ dog	old ⊙ cat
runs	1	4	0	1	4	0	0
barks	5	0	7	12	7	35	0

- Correlate with human similarity judgments about adjective-noun, noun-noun, verb-noun and noun-verb pairs
- The additive and the multiplicative model are **symmetric** (commutative): They do not take word order or syntax into account.
  - *John hit the ball = The ball hit John*
- More suitable for modeling **content words**, would not apply well to function words (e.g. conjunctions, prepositions etc.):
  - some dogs, lice and dogs, lice on dogs

LEXICAL FUNCTION MODELS

Distinguish between:

- words whose meaning is directly determined by their distributional profile, e.g. nouns
- words that act as **functions** transforming the distributional profile of other words, e.g., adjectives, adverbs



LEXICAL FUNCTION MODELS

Baroni and Zamparelli. (2010). Nouns are vectors, adjectives are matrices: Representing adjective-noun constructions in semantic space. In *Proceedings of EMNLP*.

Adjectives modelled as **lexical functions** that are applied to nouns: *old dog = old(dog)*

LEXICAL FUNCTION MODELS

Baroni and Zamparelli. (2010). Nouns are vectors, adjectives are matrices: Representing adjective-noun constructions in semantic space. In *Proceedings of EMNLP*.

Adjectives modelled as **lexical functions** that are applied to nouns: *old dog = old(dog)*

- Adjectives are parameter matrices ($\mathbf{A}_{old}$, $\mathbf{A}_{big}$, etc.)
- Nouns are vectors (**house**, **dog**, etc.)
- Composition is a linear transformation: **old dog** = $\mathbf{A}_{old}$ **x dog**.

LEXICAL FUNCTION MODELS

Baroni and Zamparelli. (2010). Nouns are vectors, adjectives are matrices: Representing adjective-noun constructions in semantic space. In *Proceedings of EMNLP*.

Adjectives modelled as **lexical functions** that are applied to nouns: *old dog = old(dog)*

- Adjectives are parameter matrices ($\mathbf{A}_{old}$, $\mathbf{A}_{big}$, etc.)
- Nouns are vectors (**house**, **dog**, etc.)
- Composition is a linear transformation: **old dog = $\mathbf{A}_{old}$ x dog**.

OLD	runs	barks
runs	0.5	0
barks	0.3	1

x

	dog
runs	1
barks	5

=

	OLD(dog)
runs	(0.5 x 1) + (0 x 5) = 0.5
barks	(0.3 x 1) + (5 x 1) = 5.3

LEARNING ADJECTIVE MATRICES

For each adjective, learn a parameter matrix that allows to predict adjective–noun phrase vectors.

	X	Y
Training set	**house**	**old house**
	dog	**old dog**
	car	**old car**
	cat	**old cat**
	toy	**old toy**
	…	…
Test set		
	elephant	**old elephant**
	mercedes	**old mercedes**

LEARNING ADJECTIVE MATRICES

1. Obtain a distributional vector $\mathbf{n}_j$ for each noun n_j in the lexicon.
2. Collect adjective noun pairs (a_i, n_j) from the corpus.
3. Obtain a distributional vector $\mathbf{p}_{ij}$ of each pair (a_i, n_j) from the same corpus using a conventional DSM.

LEARNING ADJECTIVE MATRICES

1. Obtain a distributional vector $\mathbf{n}_j$ for each noun n_j in the lexicon.
2. Collect adjective noun pairs (a_i, n_j) from the corpus.
3. Obtain a distributional vector $\mathbf{p}_{ij}$ of each pair (a_i, n_j) from the same corpus using a conventional DSM.
4. The set of tuples $\{(\mathbf{n}_j, \mathbf{p}_{ij})\}_j$ represents a dataset $D(a_i)$ for the adjective a_i.
5. Learn matrix $\mathbf{A}_i$ from $D(a_i)$ using linear regression.

Minimize the squared error loss.

$$L(\mathbf{A}_i) = \sum_{j \in D(a_i)} \left\| \mathbf{p}_{ij} - \mathbf{A}_i \mathbf{n}_j \right\|^2$$

OUTLINE

- Compositional semantics
- Compositional distributional semantics
- **Compositional semantics with neural networks**

1. How do we learn a (task-specific) **representation** of a **sentence** with a **neural network**?

2. How do we make a **prediction** for a given **task** from that representation?

TASK

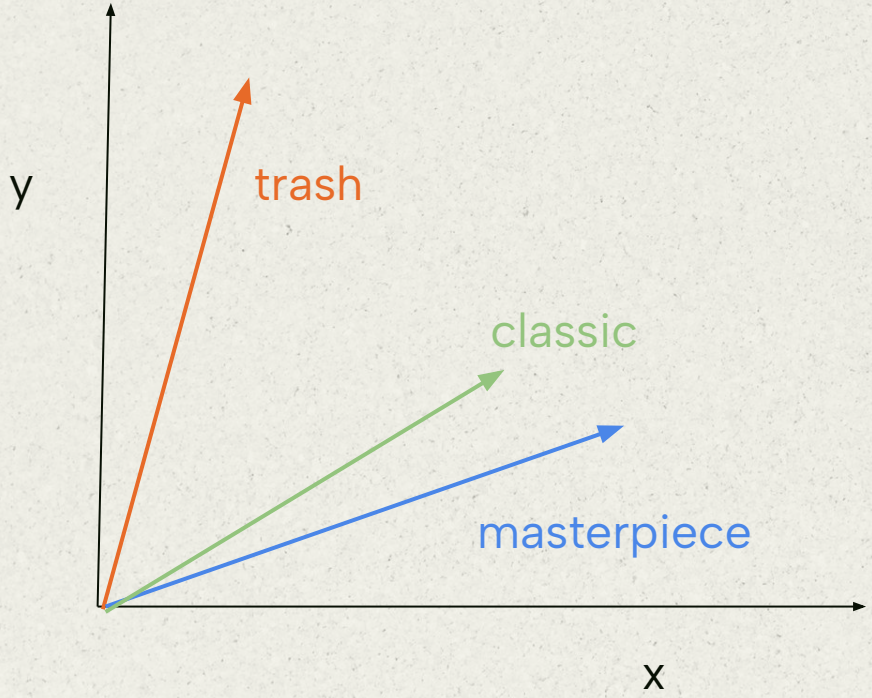
TASK: SENTIMENT CLASSIFICATION OF MOVIE REVIEWS

You'll probably love it. ->

0. Very negative
1. Negative
2. Neutral
3. **Positive**
4. Very positive

Task-specific: The learned representation has to be "specialized" on **sentiment**!

WORDS (AND SENTENCES) INTO VECTORS



WORDS (AND SENTENCES) INTO VECTORS



SENTENCE REPRESENTATION: A (VERY) SIMPLIFIED PICTURE

cDSMs (sum)

you
will
probably
love
it

NNs

you
will
probably
love
it

you will probably love it

you will probably **love** it

DATASET

DATASET: STANFORD SENTIMENT TREEBANK (SST)

Example dataset for this lecture

~12K data-points including:

1. one-sentence review + “global” sentiment score
2. tree structure (syntax)
3. more detailed sentiment scores (node-level)

MODELS

MODELS

1. Bag of Words (BOW)
2. Continuous Bag of Words (CBOW)
3. Deep Continuous Bag of Words (Deep CBOW)
4. Deep CBOW + pre-trained word embeddings
5. LSTM
6. Tree LSTM

FIRST APPROACH: SENTENCE + SENTIMENT

1. **one-sentence review + “global” sentiment score**
2. tree structure (syntax)
3. node-level sentiment scores

1. BAG OF WORDS (BOW)

WHAT IS A BAG OF WORDS?

- Additive model: does not take word order or syntax into account
- Task-specific word representations with **fixed dimensionality** (d=5)
- Dimensions of vector space are explicit, **interpretable**



Credits: CMU

BAG OF WORDS

Sum word embeddings, add bias

I

loved

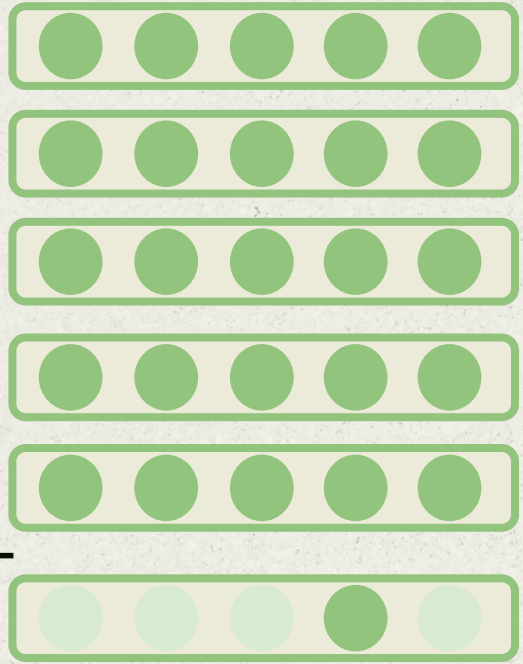
this

movie

bias **b**

——————

$\Sigma x_t + b$



argmax **3**

BAG OF WORDS

this	[0.0, 0.1, 0.1, 0.1, 0.0]
movie	[0.0, 0.1, 0.1, 0.2, 0.1]
is	[0.0, 0.1, 0.0, 0.0, 0.0]
stupid	[0.9, 0.5, 0.1, 0.0, 0.0]
bias	[0.0, 0.0, 0.0, 0.0, 0.0]
sum	[0.9, 0.8, 0.3, 0.3, 0.1]

argmax: 0 (very negative)

BAG OF WORDS

this	[0.0, 0.1, 0.1, 0.1, 0.0]
movie	[0.0, 0.1, 0.1, 0.2, 0.1]
is	[0.0, 0.1, 0.0, 0.0, 0.0]
stupid	[0.9, 0.5, 0.1, 0.0, 0.0]
bias	[0.0, 0.0, 0.0, 0.0, 0.0]
————————————————————	
sum	[0.9, 0.8, 0.3, 0.3, 0.1]

argmax: 0 (very negative)

I hate that I love this movie = I love that I hate this movie

TURNING WORDS INTO NUMBERS

We want to **feed words** to a neural network

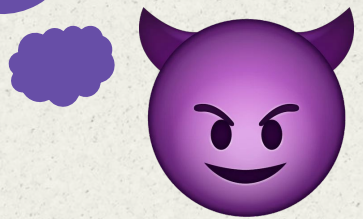
How to turn **words** into **numbers**?

Bad idea: number sequence

cat	1
tree	2
chair	3
dog	4
mat	5

Good idea: one-hot vectors

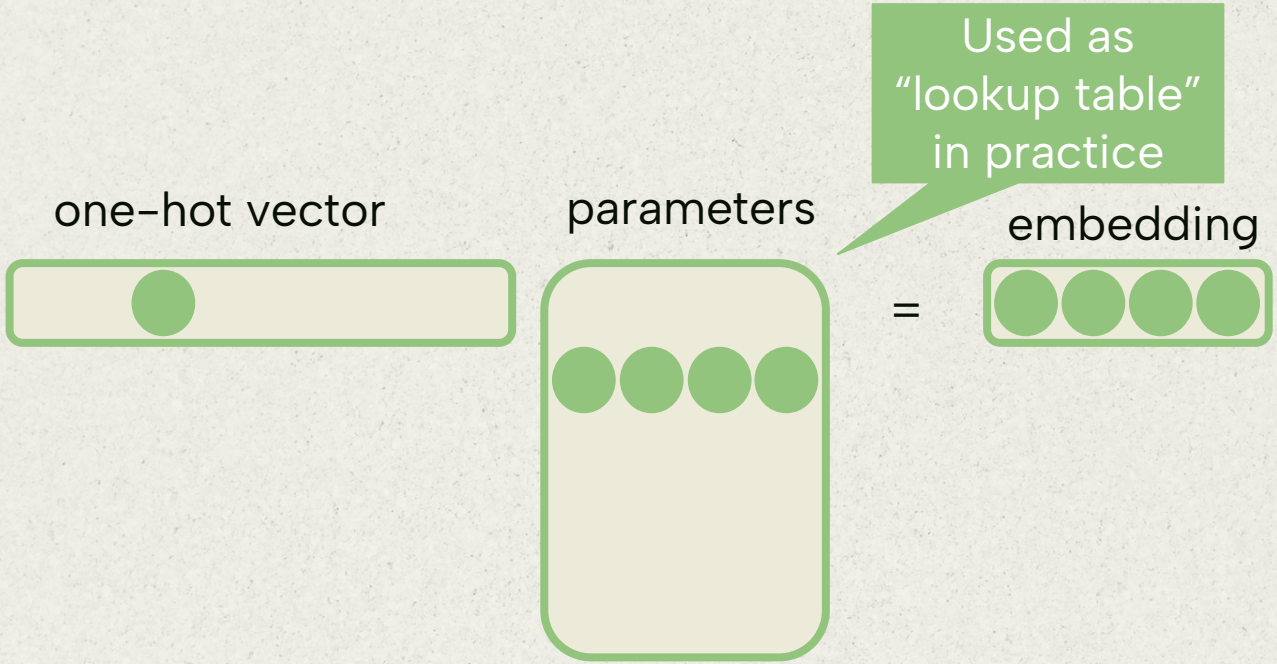
cat	[0,0,0,0,1]
tree	[0,0,0,1,0]
chair	[0,0,1,0,0]
dog	[0,1,0,0,0]
mat	[1,0,0,0,0]





cat is closer to **tree** than to **dog**?!

ONE-HOT VECTORS SELECT WORD EMBEDDINGS



2. CONTINUOUS BAG OF WORDS (CBOW)

CBOW

- Additive model: does not take word order or syntax into account
- Task-specific word representations of **arbitrary dimensionality**
- Dimensions of vector space are **not interpretable**
- Prediction can be traced back to the sentence vector dimensions

CONTINUOUS BAG OF WORDS (CBOW)

Sum word embeddings, project to 5D using W, add bias: $W(\mathbf{\Sigma x_t}) + \mathbf{b}$

I



loved



this



movie



——————

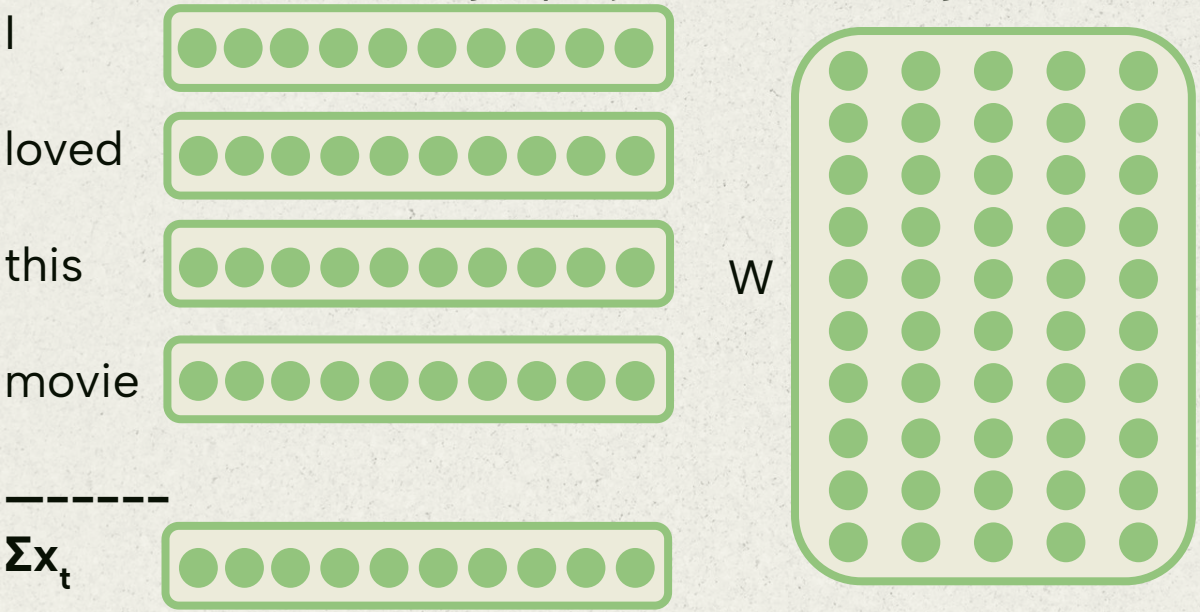
$\mathbf{\Sigma x_t}$



Note that a bias term (of size 5) is added to the final output vector (not shown). Also, this is not the same as word2vec CBOW!

CONTINUOUS BAG OF WORDS (CBOW)

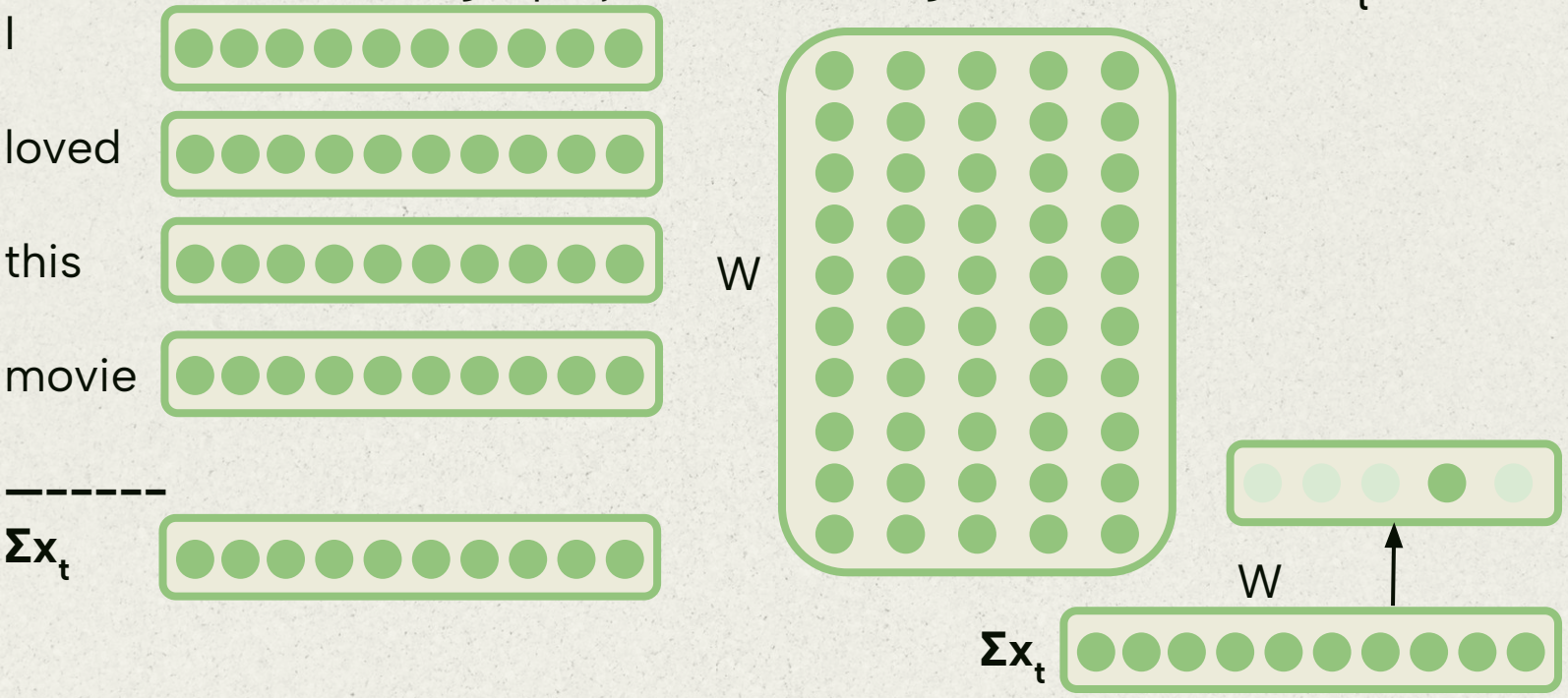
Sum word embeddings, project to 5D using W, add bias: $W(\mathbf{\Sigma x_t}) + \mathbf{b}$



Note that a bias term (of size 5) is added to the final output vector (not shown). Also, this is not the same as word2vec CBOW!

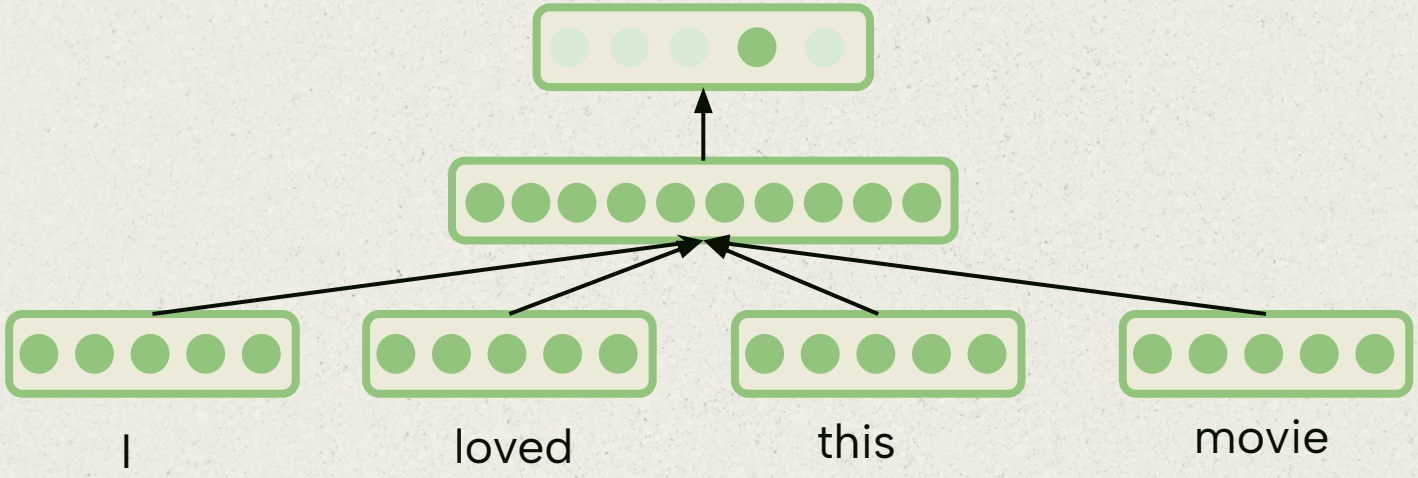
CONTINUOUS BAG OF WORDS (CBOW)

Sum word embeddings, project to 5D using W, add bias: $W(\Sigma x_t) + \mathbf{b}$

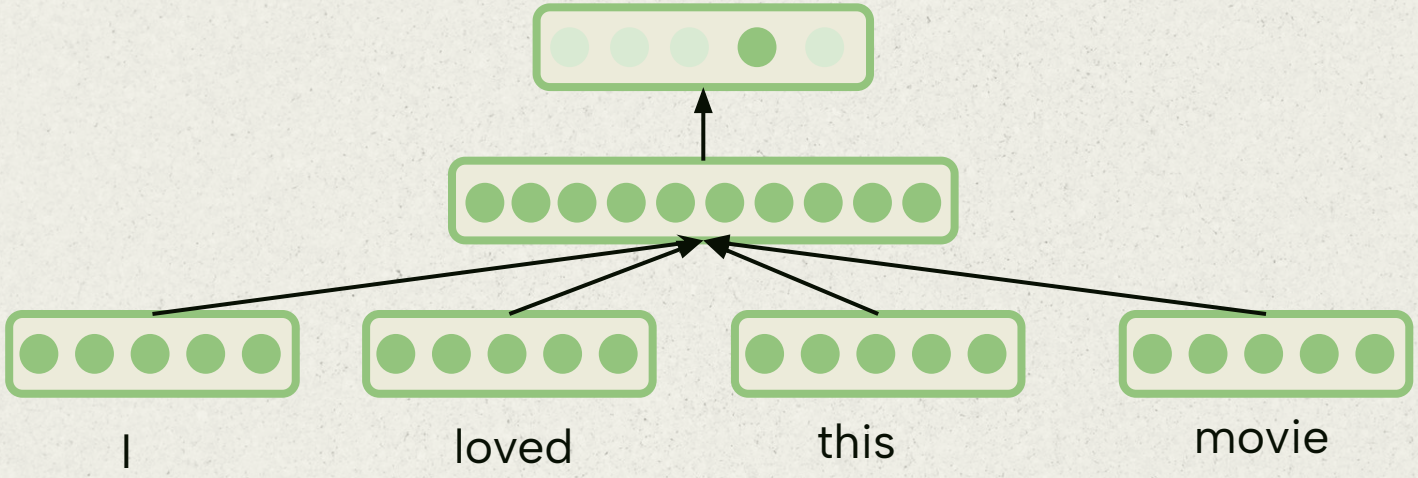


Note that a bias term (of size 5) is added to the final output vector (not shown). Also, this is not the same as word2vec CBOW!

WHAT ABOUT THIS?



WHAT ABOUT THIS?



Variable sentence vector size, dependent on sentence length

- Not very sensible conceptually
  - sentences in a different vector space than words
  - one vector space for each sentence length in the dataset
- Difficult in practice
  - what size should the transformation matrix be?
  - vector size can grow very large

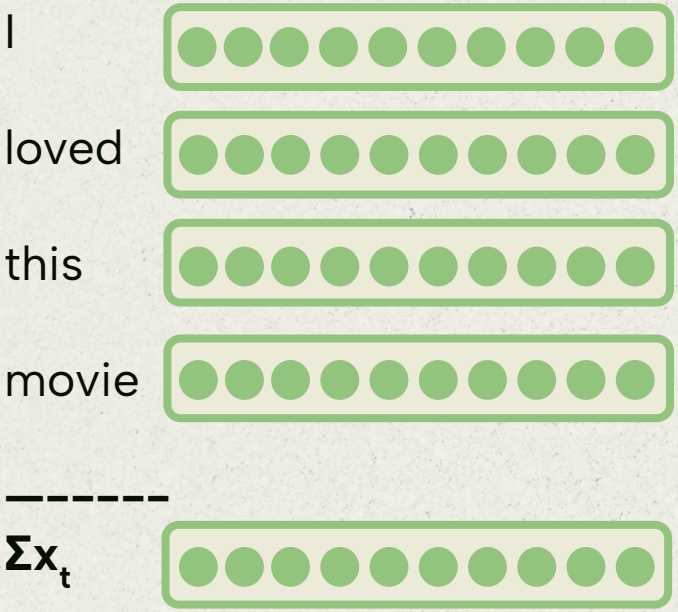
3. DEEP CBOW

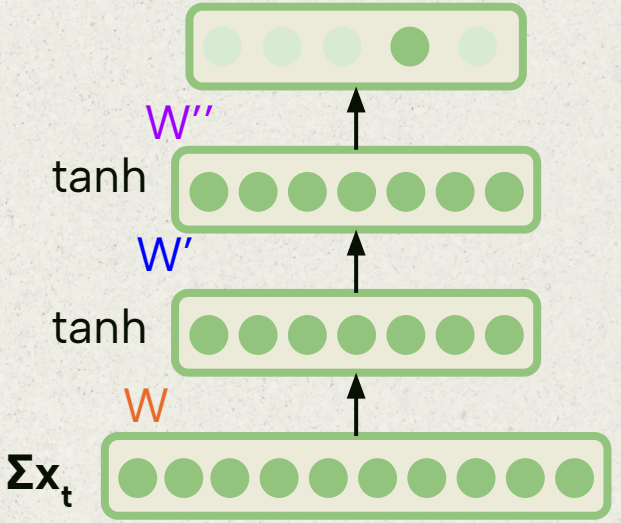
DEEP CBOW

- Additive model: does not take word order or syntax into account
- Task-specific word representations of **arbitrary dimensionality**
- Dimensions of vector space are **not interpretable**
- **More layers and non-linear transformations**: prediction cannot be easily traced back

DEEP CBOW

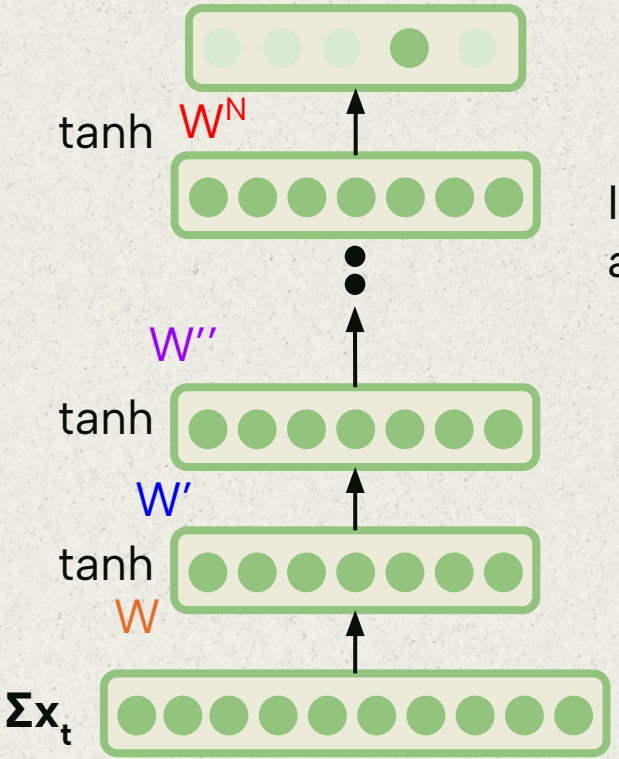
$W'' \tanh(W' \tanh(W(\Sigma x_t) + b) + b') + b'')$





Note that a bias term is added whenever we multiply with a W (not shown)

WHAT ABOUT THIS?



Is more complexity always better?

QUESTION

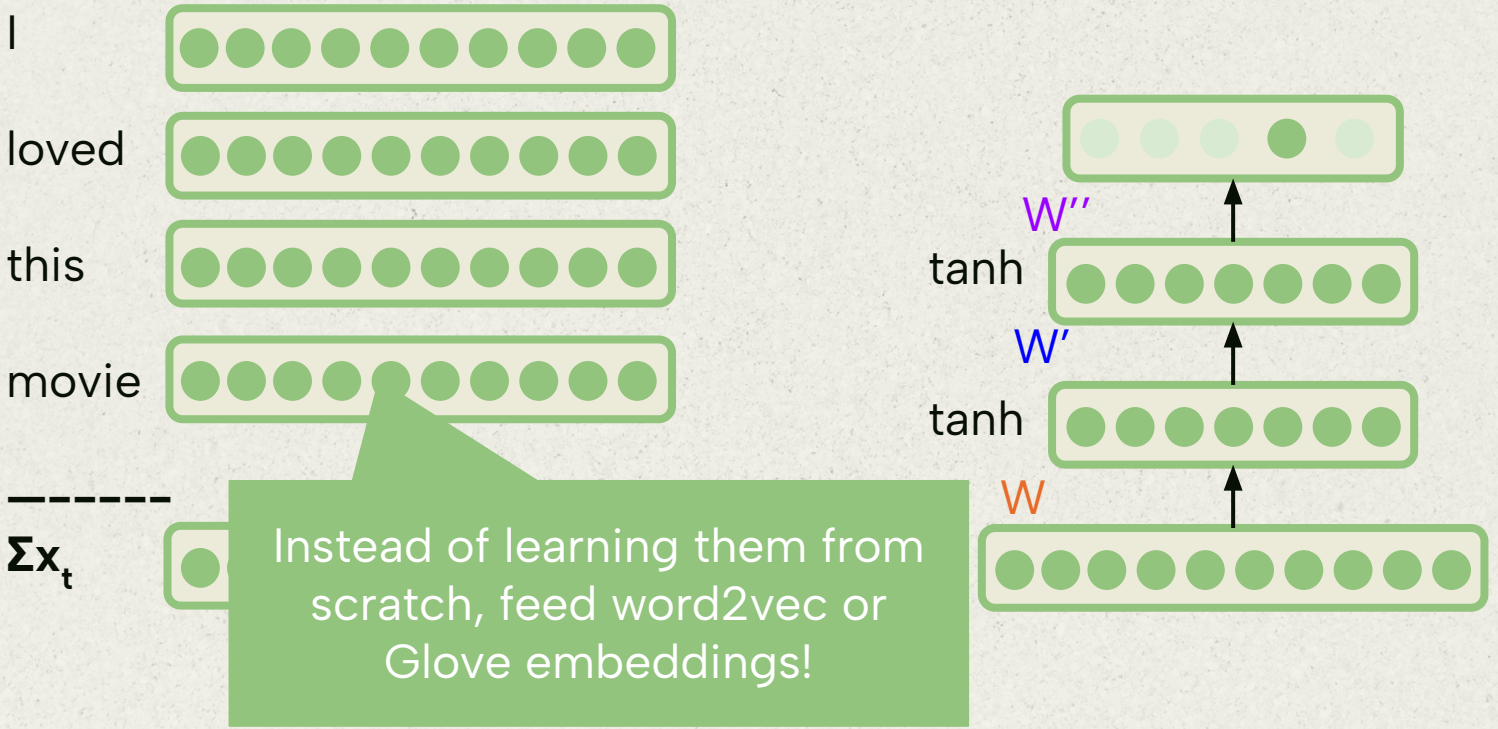
We can learn more complex features, but the only error signal that we receive comes from sentiment prediction.

How can we further help the model?

4. DEEP CBOW + PRETRAINED EMBEDDINGS

DEEP CBOW WITH PRETRAINED EMBEDDINGS

$W'' \tanh(W' \tanh(W(\Sigma x_t) + b) + b') + b'')$



Note that a bias term is added whenever we multiply with a W (not shown)

DEEP CBOW + PRE-TRAINED EMBEDDINGS

- Additive model: does not take word order or syntax into account
- Dimensions of vector space are **not interpretable**
- Multiple layers and non-linear transformations: prediction cannot be easily traced back
- Pre-trained **general-purpose** word representations (e.g., Skip-gram, GloVe)
  - **keep frozen:** not updated during training
  - **fine-tune:** updated with task-specific learning signal (specialized)

RECAP: TRAINING A NEURAL NETWORK

We train our network with Stochastic Gradient Descent (SGD):

1. Sample a training example
2. Forward pass
   a. Compute network activations, output vector
3. Compute loss
   a. Compare output vector with true label using a **loss function (Cross Entropy)**
4. Backward pass (backpropagation)
   a. Compute gradient of loss w.r.t. (learnable) parameters (= weights + bias)
5. Take a small step in the opposite direction of the gradient

CROSS ENTROPY LOSS

Given:

$\hat{Y}$ = [0.0589, 0.0720, 0.0720, **0.7177**, 0.0795]
output vector (after softmax) from forward pass

Y = [0, 0, 0, 1, 0] target / label (y_3=1)

When our output is categorical (i.e., a number of classes), we can use a Cross Entropy loss:

$CE(\mathbf{y},\hat{\mathbf{y}}) = -\Sigma\, y_i \log\ \hat{y}_i$

$SparseCE(y=3, \hat{\mathbf{y}}) = -\log \hat{y}_y$

CROSS ENTROPY LOSS

Given:

$\hat{Y}$ = [0.0589, 0.0720, 0.0720, **0.7177**, 0.0795]
output vector (after softmax) from forward pass

Y = [0, 0, 0, 1, 0] target / label (y_3=1)

When our output is categorical (i.e., a number of classes), we can use a Cross Entropy loss:

$CE(\mathbf{y},\hat{\mathbf{y}}) = -\Sigma\, y_i \log\ \hat{y}_i$

$SparseCE(y=3, \hat{\mathbf{y}}) = -\log \hat{y}_y$

torch.nn.CrossEntropyLoss works like this and does the **softmax** on **o** for you!

SOFTMAX

We don't need a softmax for **prediction**, there we simply take the **argmax**

o = [-0.1, 0.1, 0.1, **2.4**, 0.2]

$$\text{softmax}(o_i) = \exp(o_i) / \Sigma_j \exp(o_j)$$

This makes **o** sum to 1.0:
softmax(**o**) = [0.0589, 0.0720, 0.0720, **0.7177**, 0.0795]

But we do need a **softmax** combined to CE to compute model loss (argmax is NOT differentiable)

BREAK

RECURRENT NEURAL NETWORKS

INTRODUCTION: RECURRENT NEURAL NETWORK (RNN)

- RNNs widely used for handling **sequences**!
- RNNs ~ **multiple copies of same network**, each passing a message to a successor
- Take an input vector *x* and output an output vector *h*
- Crucially, *h* **influenced by entire history** of inputs fed in in the past
- Internal state *h* gets updated at every time step –> in the simplest case, this state consists of a **single hidden vector *h***

Elman, J. L. (1990). Finding structure in time. *Cognitive Science, 14*(2), 179-211.

INTRODUCTION: RECURRENT NEURAL NETWORK (RNN)

RNNs model **sequential data** – one input $\mathbf{x}_t$ per time step *t*

Example:

the	cat	sat	on	the	mat
$\mathbf{x}_1$	$\mathbf{x}_2$	$\mathbf{x}_3$	$\mathbf{x}_4$	$\mathbf{x}_5$	$\mathbf{x}_6$

Let's compute the RNN state after reading in this sentence.

Remember:

$\mathbf{h}_t = f(\mathbf{x}_t, \mathbf{h}_{t-1})$

$h_1 = f(x_1, h_0)$
$h_2 = f(x_2, f(x_1, h_0))$
$h_3 = f(x_3, f(x_2, f(x_1, h_0)))$
...
$h_6 = f(x_6, f(x_5, f(x_4, ...)))$

INTRODUCTION: RECURRENT NEURAL NETWORK (RNN)

RNNs model **sequential data** – one input $\mathbf{x}_t$ per time step *t*

Example:

the cat sat on the mat

$\mathbf{x}_1$ $\mathbf{x}_2$ $\mathbf{x}_3$ $\mathbf{x}_4$ $\mathbf{x}_5$ $\mathbf{x}_6$

Let's compute the RNN state after reading in this sentence.

Remember:

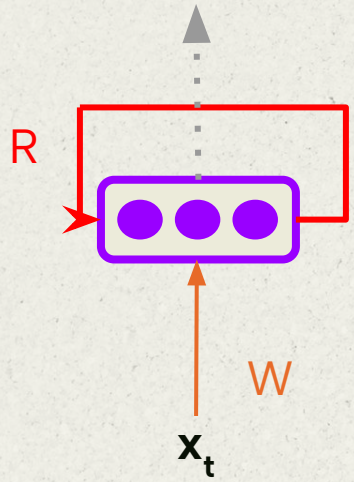
$\mathbf{h}_t = f(\mathbf{x}_t, \mathbf{h}_{t-1})$

$h_1 = f(x_1, h_0)$
$h_2 = f(x_2, f(x_1, h_0))$
$h_3 = f(x_3, f(x_2, f(x_1, h_0)))$
...
$h_6 = f(x_6, f(x_5, f(x_4, ...)))$

the -> $h_1 = f(x_1, h_0)$
cat -> $h_2 = f(x_2, h_1)$
sat -> $h_3 = f(x_3, h_2)$
...
mat -> $h_6 = f(x_6, h_5)$

INTRODUCTION: RECURRENT NEURAL NETWORK (RNN)

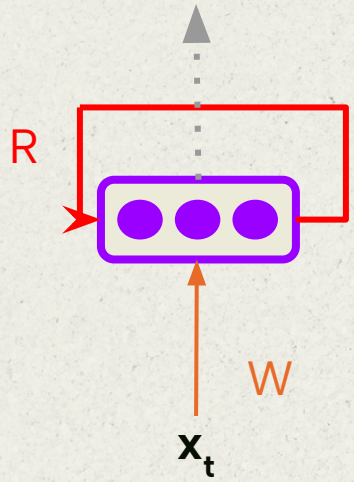
The transition function f consists of an affine transformation followed by a non-linear activation



Elman (1990). Finding structure in time.

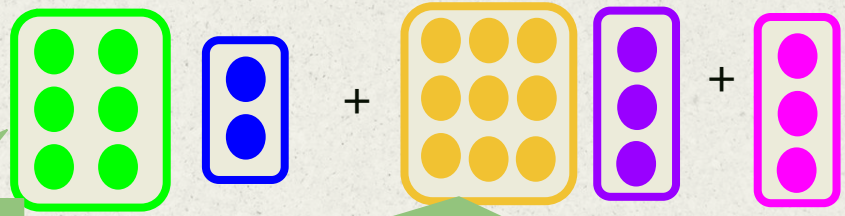
INTRODUCTION: RECURRENT NEURAL NETWORK (RNN)

The transition function f consists of an affine transformation followed by a non-linear activation



$$h_t = f(x_t, h_{t-1})$$

$$= \sigma\,(W x_t + R\, h_{t-1} + b)$$

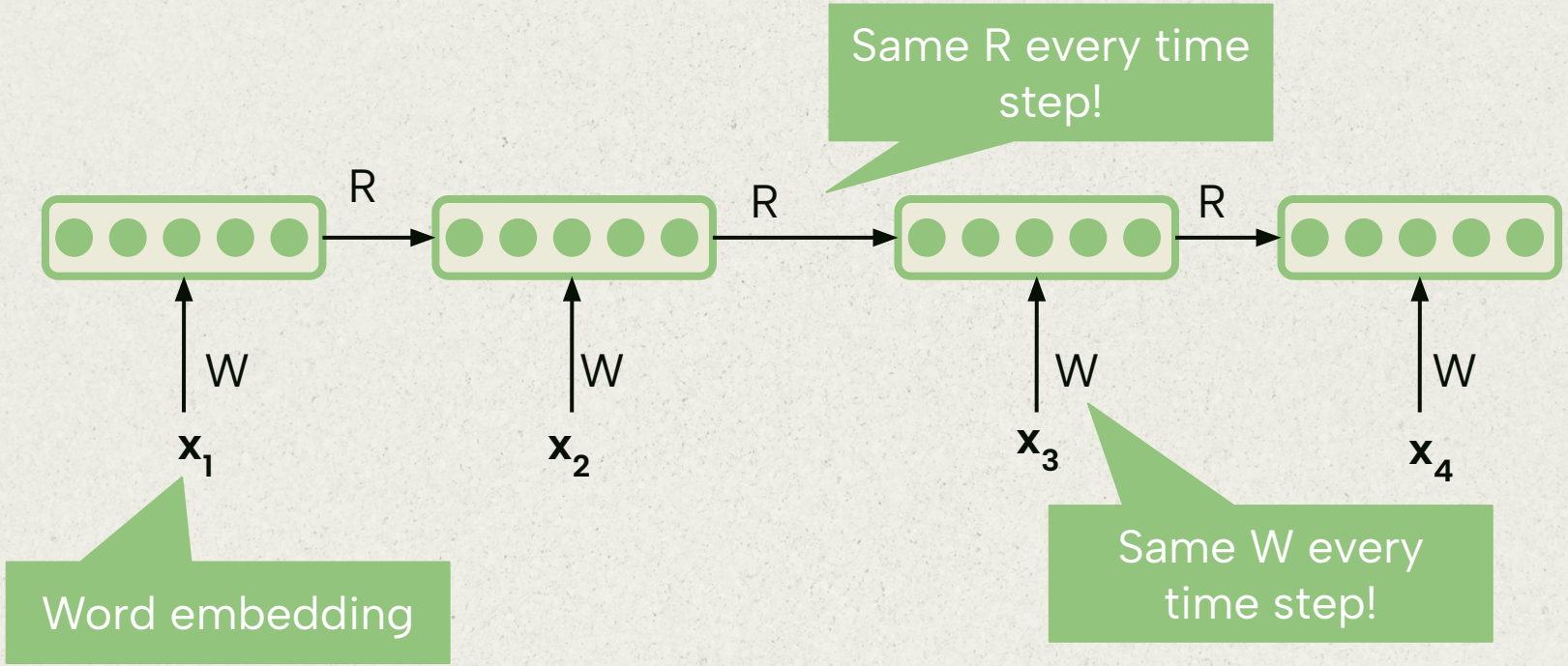


Matrix based on current input

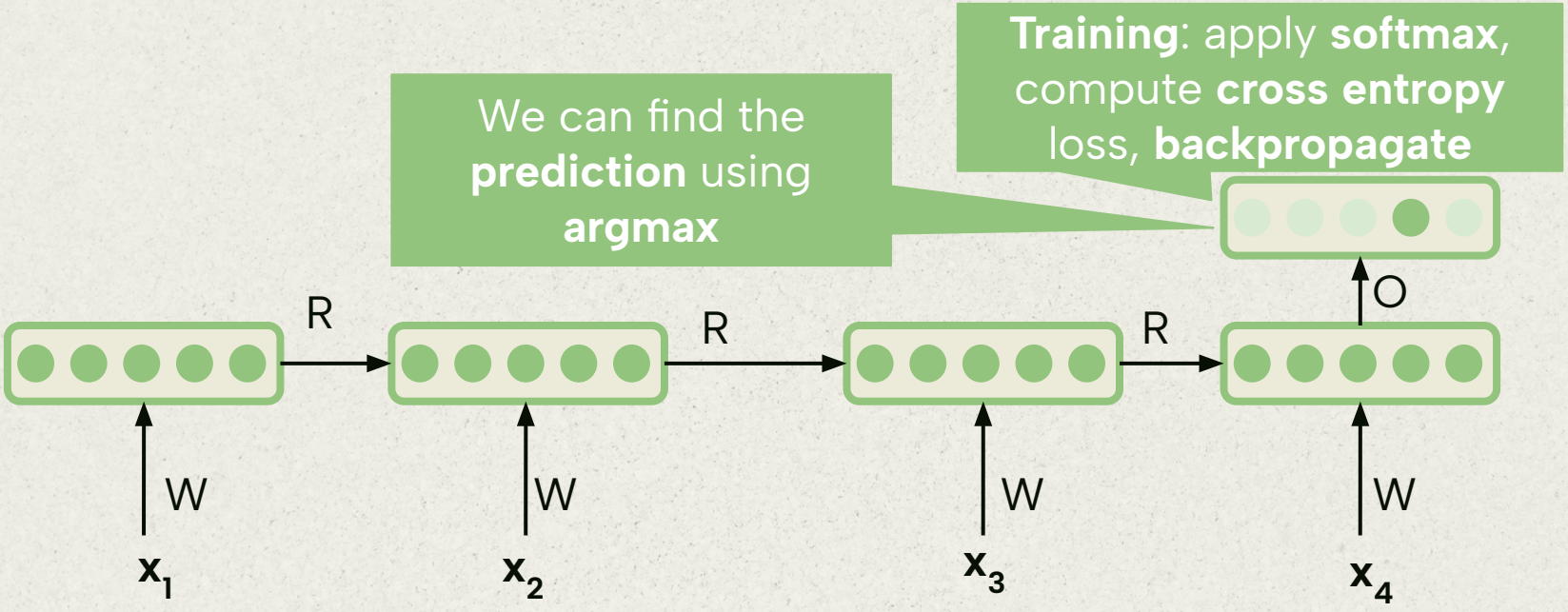
Matrix based on the previous hidden state

Elman (1990). Finding structure in time.

INTRODUCTION: UNFOLDING THE RNN



INTRODUCTION: UNFOLDING THE RNN



INTRODUCTION: THE VANISHING GRADIENT PROBLEM

Simple RNNs are hard to train because of the **vanishing gradient** problem.

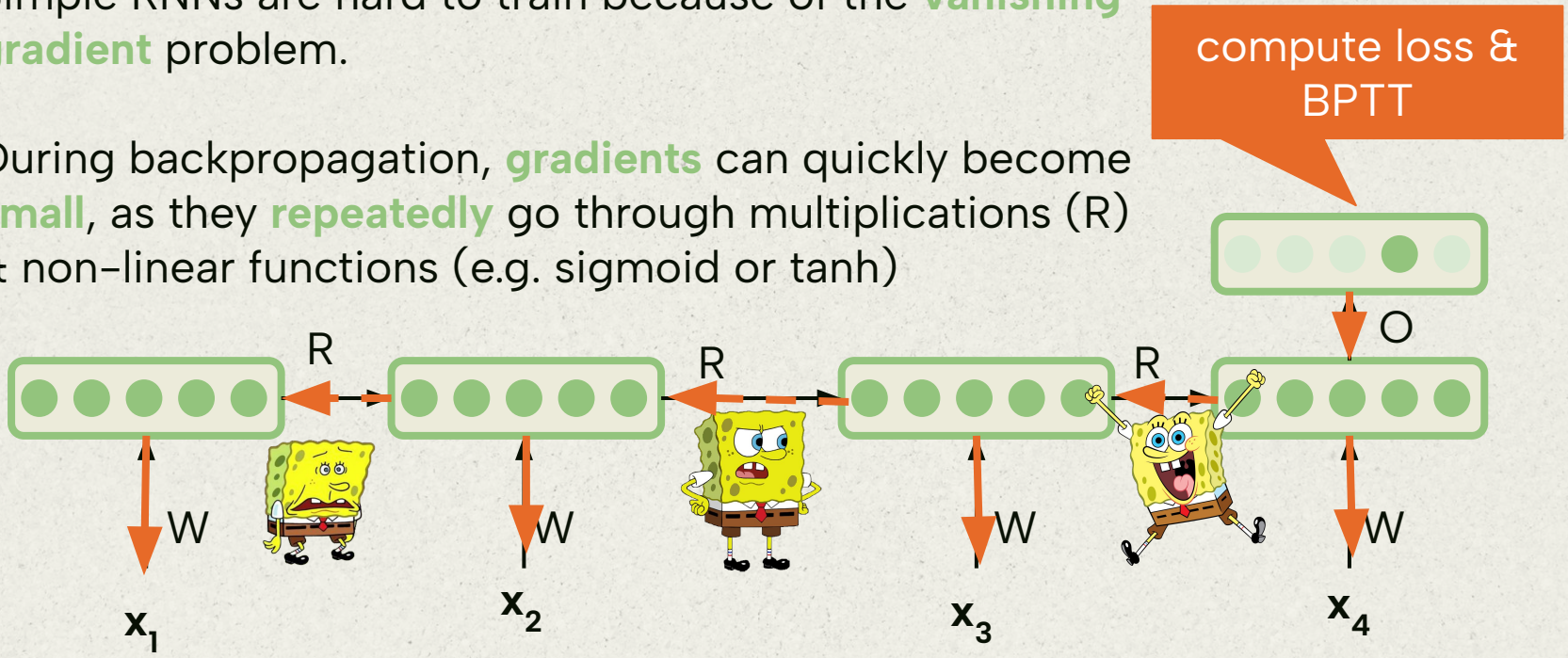
During backpropagation, **gradients** can quickly become **small**, as they **repeatedly** go through multiplications (R) & non-linear functions (e.g. sigmoid or tanh)

For more details see: Kyunghyun Cho. Natural Language Understanding with Distributed Representation. Section 4.3.

INTRODUCTION: THE VANISHING GRADIENT PROBLEM

Simple RNNs are hard to train because of the **vanishing gradient** problem.

During backpropagation, **gradients** can quickly become **small**, as they **repeatedly** go through multiplications (R) & non-linear functions (e.g. sigmoid or tanh)



For more details see: Kyunghyun Cho. Natural Language Understanding with Distributed Representation. Section 4.3.

INTRODUCTION: THE VANISHING GRADIENT PROBLEM

R is shared across every timestep!

Imagine that R contains an entry value $r_1 = 0.5$

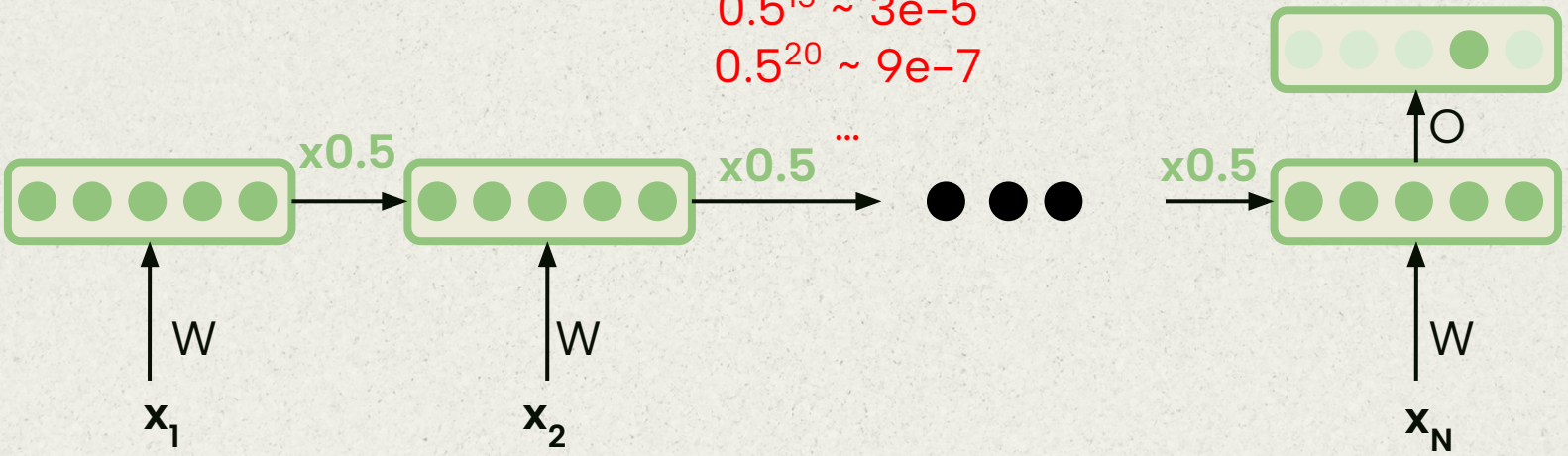
The first input gets multiplied by $\mathbf{0.5}^{\text{num. unrolls N}}$

$0.5^5 \sim 0.03$

$0.5^{10} \sim 9e{-}4$

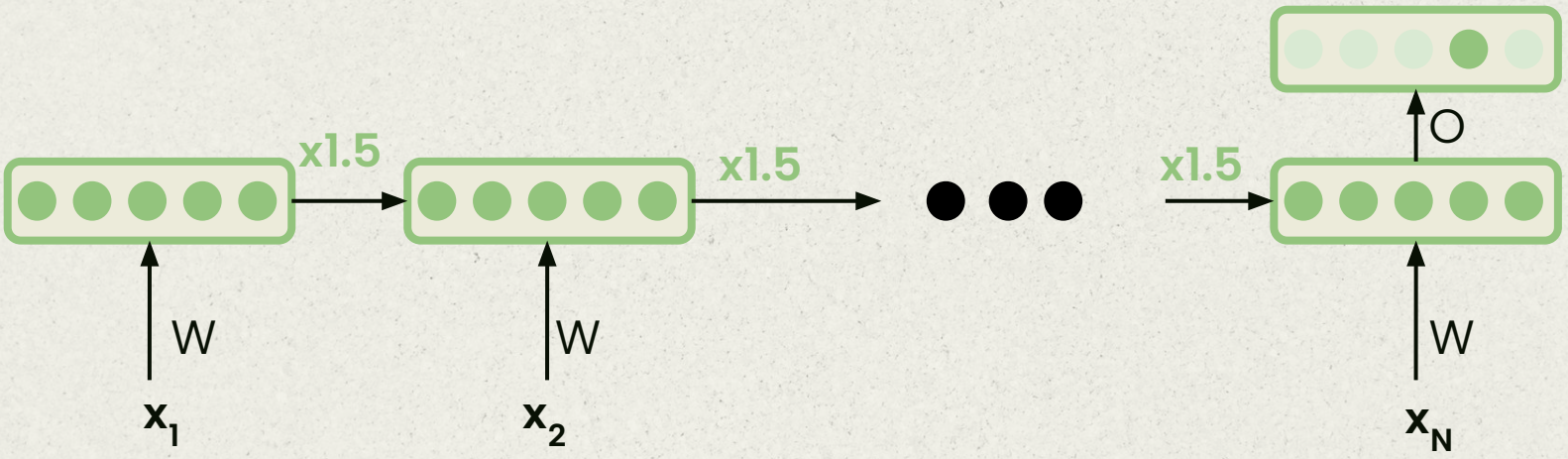
$0.5^{15} \sim 3e{-}5$

$0.5^{20} \sim 9e{-}7$



For more details see: Kyunghyun Cho. Natural Language Understanding with Distributed Representation. Section 4.3.

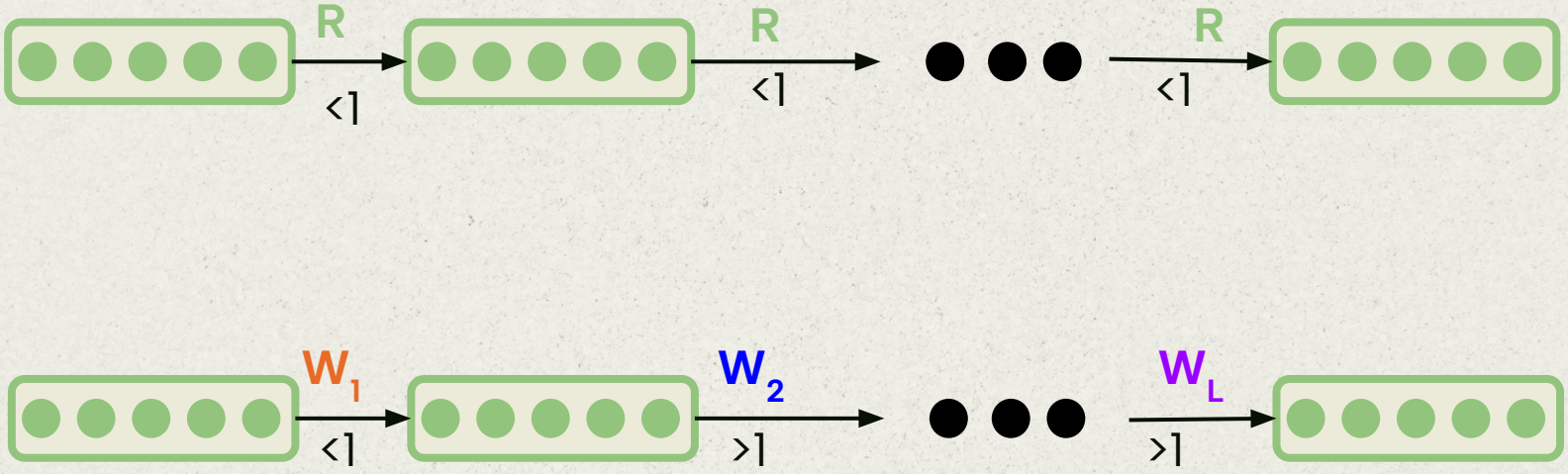
WHAT ABOUT THIS?



Similar problem called exploding gradients!

For more details see: Kyunghyun Cho. Natural Language Understanding with Distributed Representation. Section 4.3.

RNN vs ANN



5. LONG SHORT-TERM MEMORY NETWORK (LSTM)

LONG SHORT-TERM MEMORY (LSTM)

LSTMs are a special kind of RNN that can deal with **long-term dependencies** in the data by alleviating the vanishing gradient problem in RNNs

“I lived in **France** for a while when I was a kid so I can speak fluent…” -> French

Hochreiter, S. & Schmidhuber, J. (1997). Long short-term memory. *Neural computation, 9*(8), 1735-1780.

LSTM: CORE IDEA

1. Maintain a **separate memory cell state** c_t from what is outputted (long term memory)

LSTM: CORE IDEA

1. Maintain a **separate memory cell state** c_t from what is outputted (long term memory)

2. Use gates to control the flow of information:
   a. **Forget** gate gets rid of irrelevant information
   b. Input gate to **store** new relevant information from the current input
   c. Selectively **update** the cell state
   d. **Output** gate returns a filtered version of the cell state

LSTM: CORE IDEA

1. Maintain a **separate memory cell state** c_t from what is outputted (long term memory)

2. Use gates to control the flow of information:
   a. **Forget** gate gets rid of irrelevant information
   b. Input gate to **store** new relevant information from the current input
   c. Selectively **update** the cell state
   d. **Output** gate returns a filtered version of the cell state

3. Backpropagation through time with partially **uninterrupted gradient flow**

LSTMS

RNN:

$\mathbf{h}_t = f(\mathbf{x}_t, \mathbf{h}_{t-1})$

$= \sigma(W\mathbf{x}_t + R\,\mathbf{h}_{t-1} + b)$

LSTM:

$\mathbf{h}_t, \mathbf{c}_t = f(\mathbf{x}_t, \mathbf{h}_{t-1}, \mathbf{c}_{t-1})$

$= lstm(\mathbf{x}_t, \mathbf{h}_{t-1}, \mathbf{c}_{t-1})$

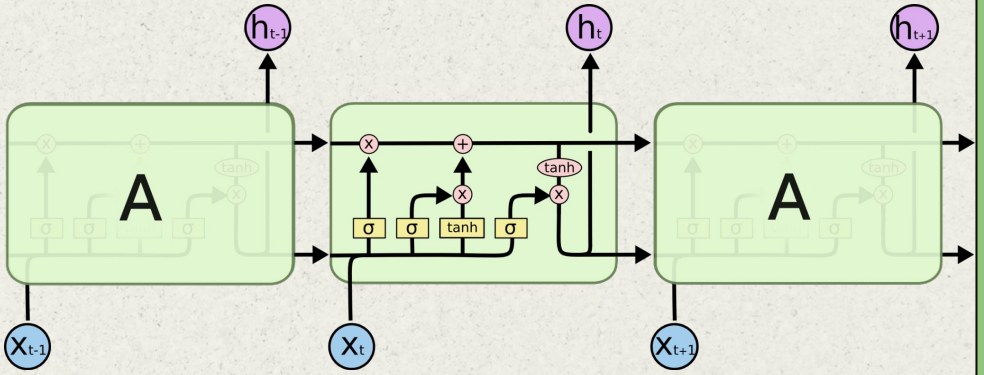
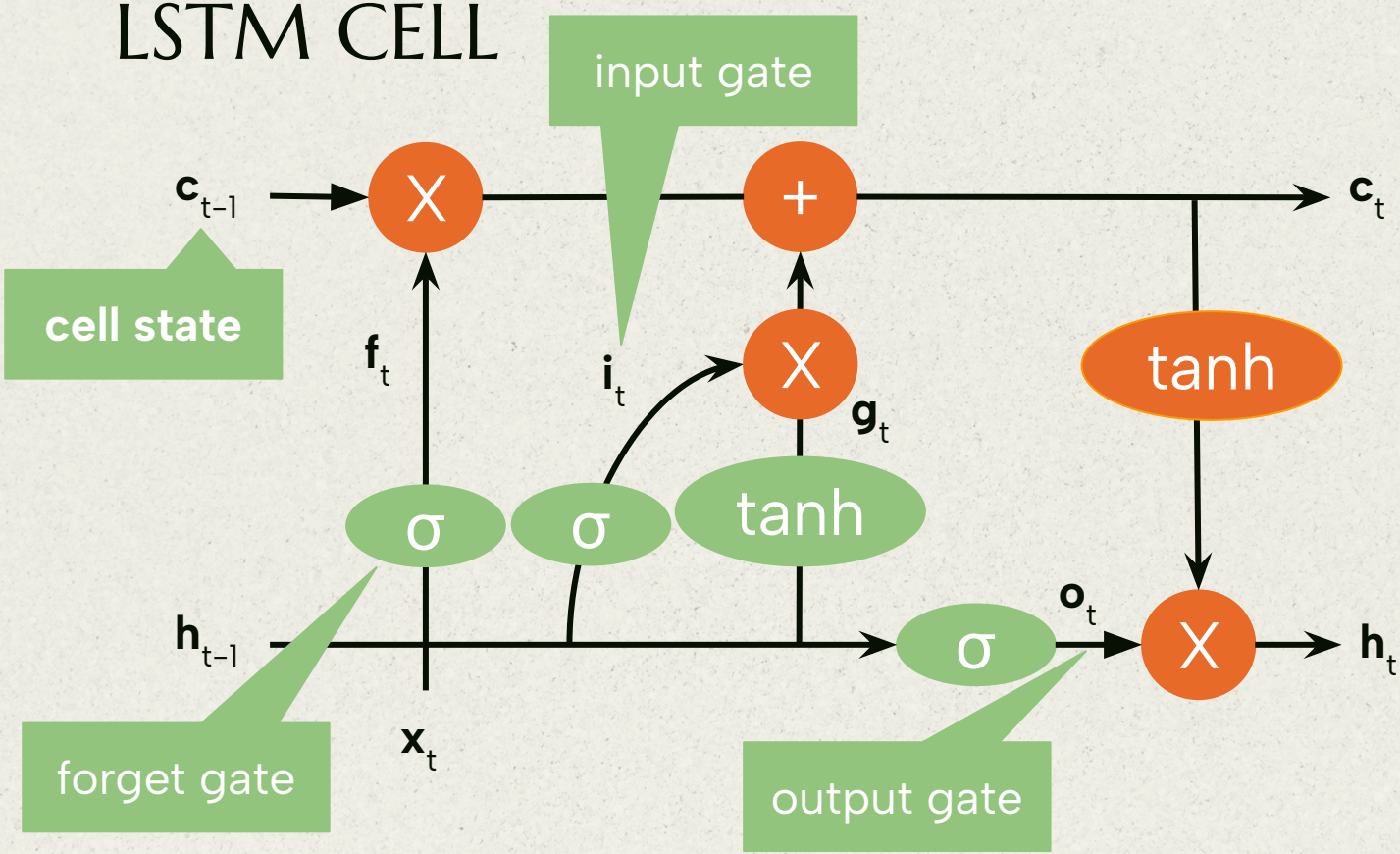


Image credits: https://colah.github.io/posts/2015-08-Understanding-LSTMs

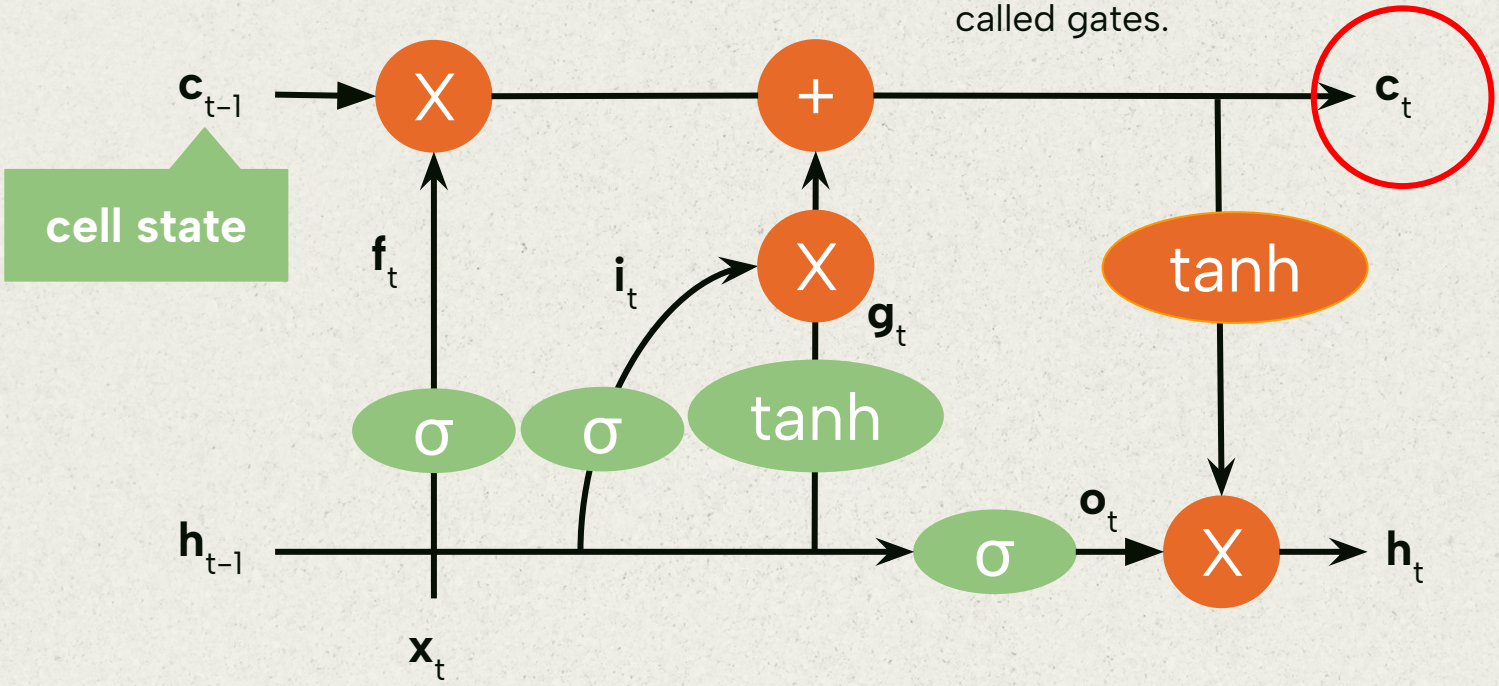
LSTM CELL



Adapted from https://colah.github.io/posts/2015-08-Understanding-LSTMs . Green blocks: $\Phi(W[\mathbf{h}_{t-1};\mathbf{x}_t]+\mathbf{b})$, orange blocks: element-wise operation

LSTM: CELL STATE

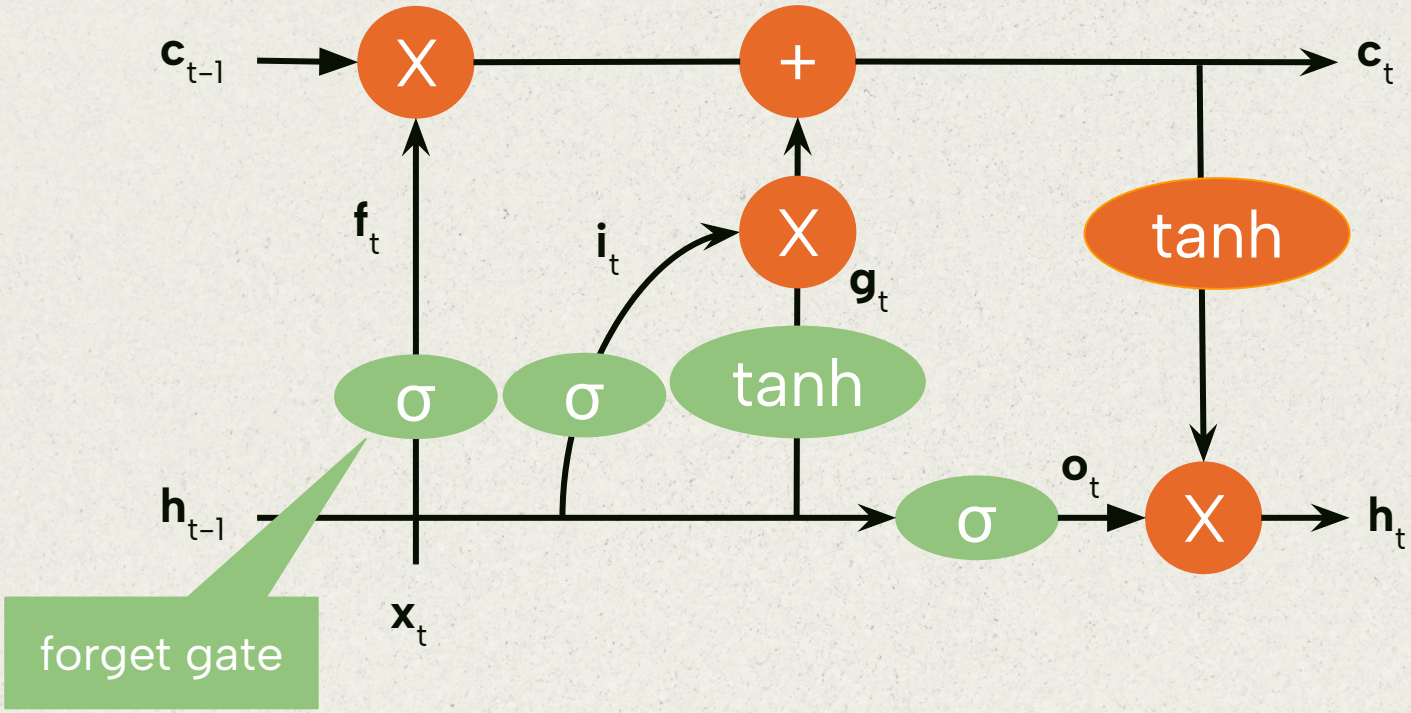
Runs straight down the entire chain, with only some minor linear interactions. LSTM can remove or add information to the cell state, carefully regulated by structures called gates.



Adapted from https://colah.github.io/posts/2015-08-Understanding-LSTMs . Green blocks: $\Phi(W[h_{t-1};x_t]+b)$, orange blocks: element-wise operation

LSTM: FORGET GATE

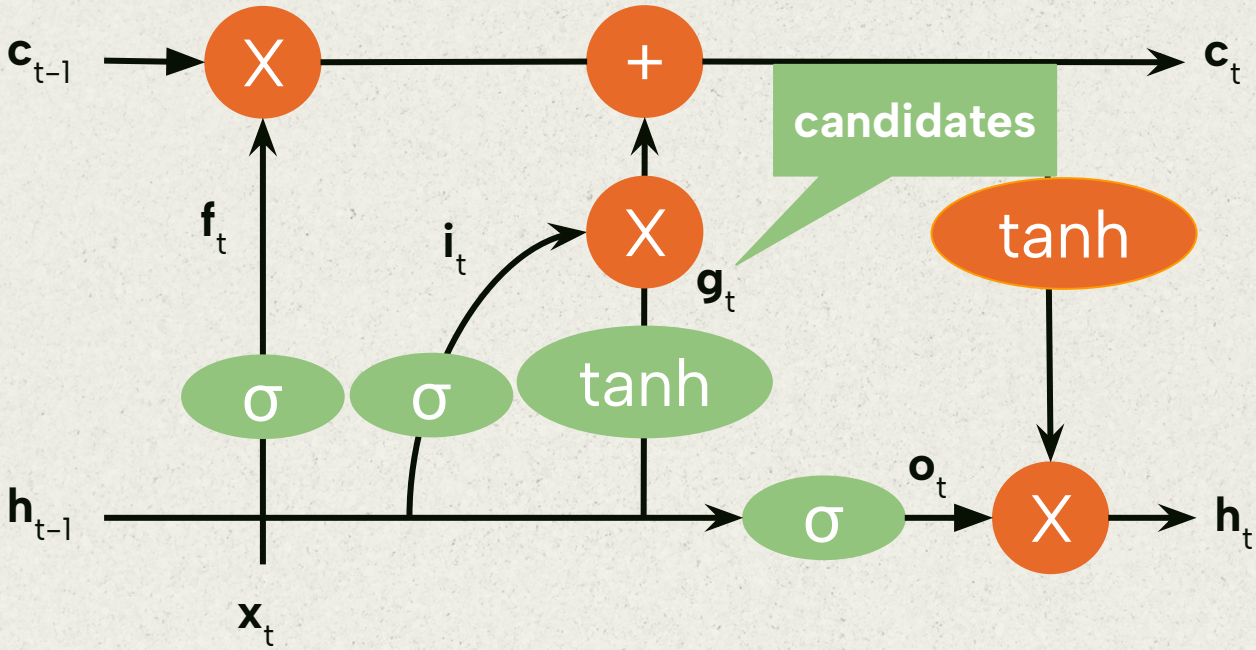
Decide what information to throw away from the cell state.



Adapted from https://colah.github.io/posts/2015-08-Understanding-LSTMs . Green blocks: $\Phi(W[h_{t-1};x_t]+b)$, orange blocks: element-wise operation

LSTM: CANDIDATE CELL

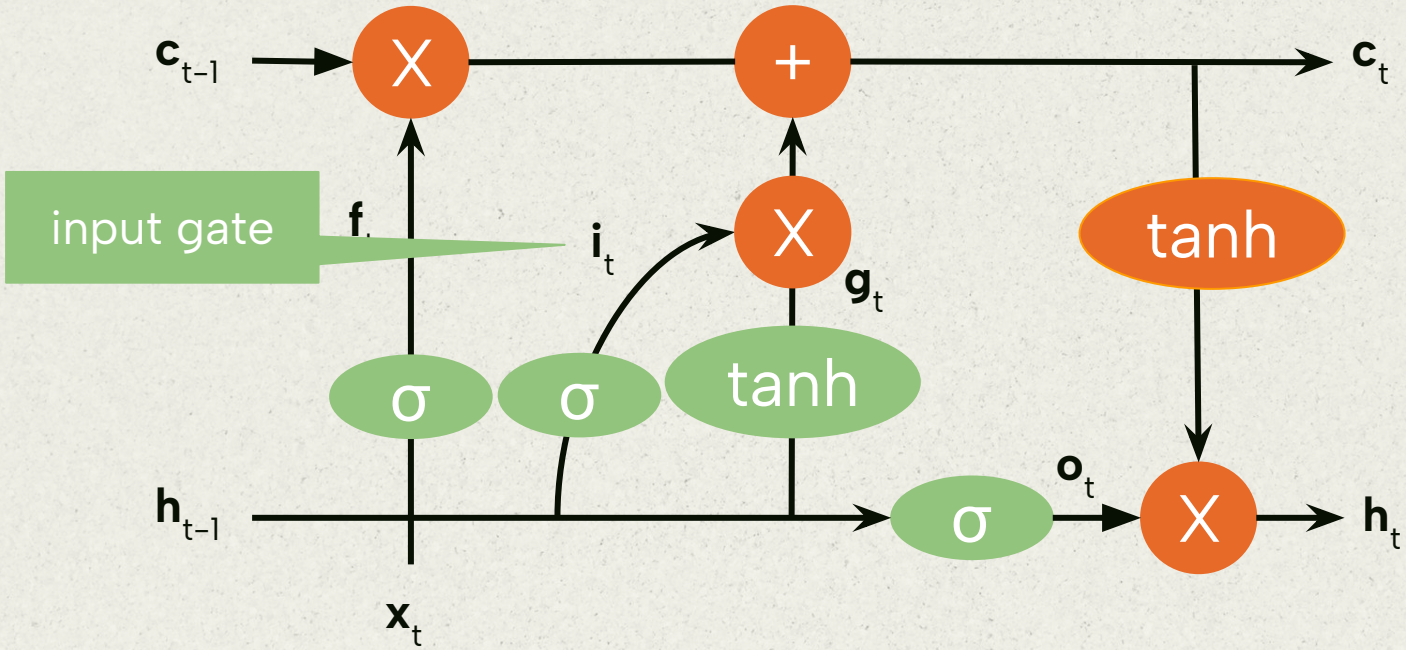
Extracts new candidate values, $\mathbf{g}_t$, from the previous hidden state and the current input that could be added to the cell state.



Adapted from https://colah.github.io/posts/2015-08-Understanding-LSTMs . Green blocks: $\Phi(W[\mathbf{h}_{t-1};\mathbf{x}_t]+\mathbf{b})$, orange blocks: element-wise operation

LSTM: INPUT GATE

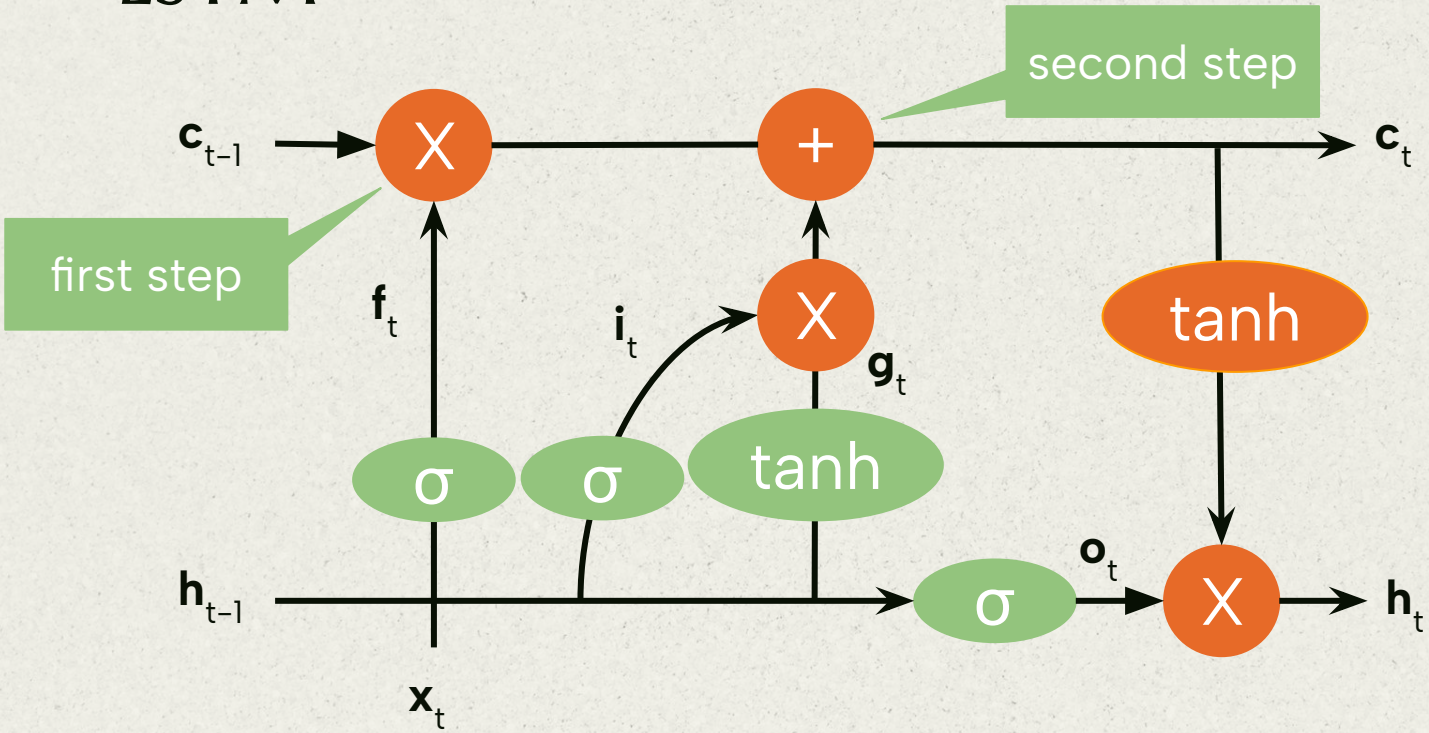
Decide what information to store in the cell state



Adapted from https://colah.github.io/posts/2015-08-Understanding-LSTMs . Green blocks: $\Phi(W[h_{t-1};x_t]+b)$, orange blocks: element-wise operation

LSTM

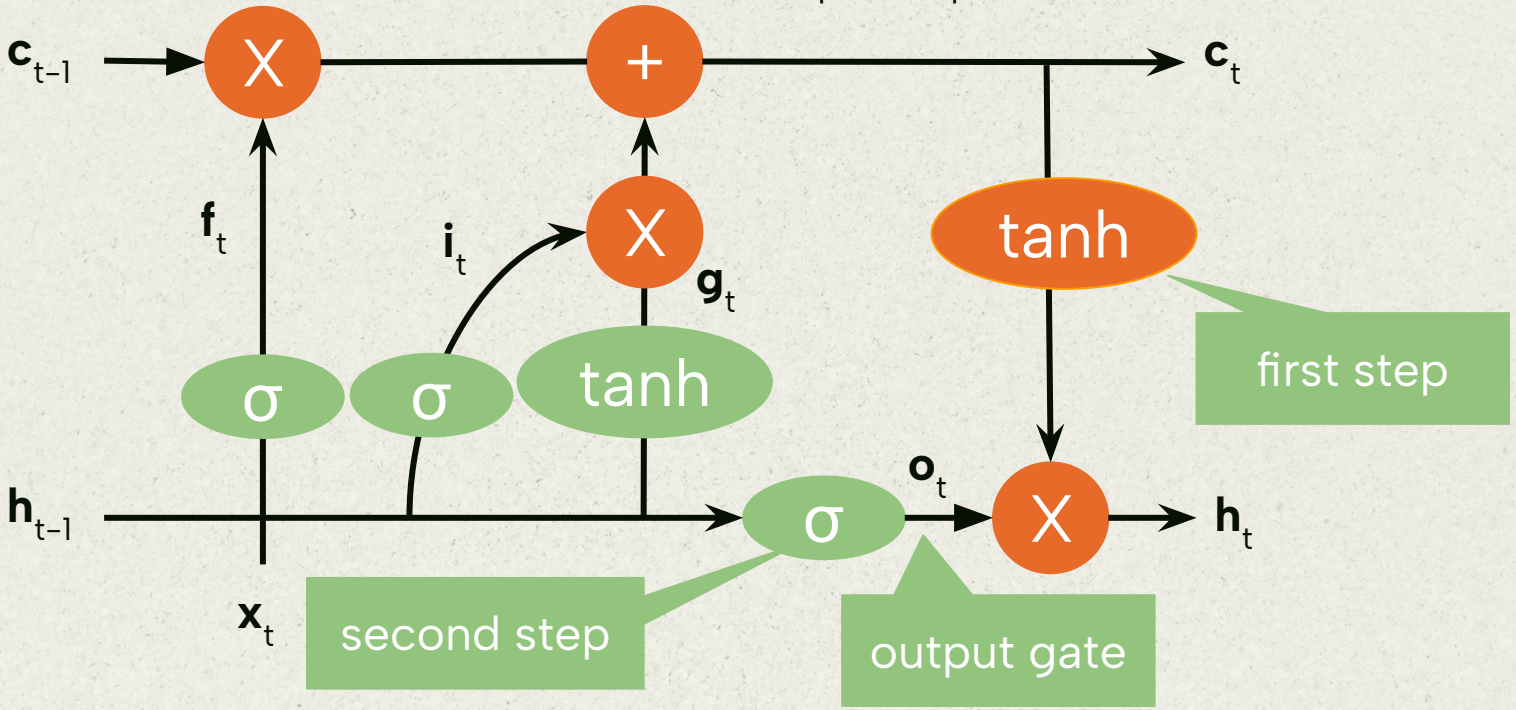
Update the cell state: 1. Forget things we decided to forget earlier, 2. Add the new candidate values scaled by how much we decided to update each state value



Adapted from https://colah.github.io/posts/2015-08-Understanding-LSTMs . Green blocks: $\Phi(W[h_{t-1};x_t]+b)$, orange blocks: element-wise operation

LSTM: OUTPUT GATE

1. Decide what parts of the cell state we are going to output, the cell state is put through *tanh* and 2. multiplied by the output of the output gate, so that we only output the parts we decided to.



Adapted from https://colah.github.io/posts/2015-08-Understanding-LSTMs . Green blocks: $\Phi(W[h_{t-1};x_t]+b)$, orange blocks: element-wise operation

LONG SHORT-TERM MEMORY (LSTM)

hidden state

cell state

previous hidden state and cell state

$$\mathbf{h}_t, \mathbf{c}_t = lstm(\mathbf{x}_t, \mathbf{h}_{t-1}, \mathbf{c}_{t-1})$$

input gate	$\mathbf{i}_t = \sigma(W_i \mathbf{x}_t + R_i \mathbf{h}_{t-1} + \mathbf{b}_i)$
forget gate	$\mathbf{f}_t = \sigma(W_f \mathbf{x}_t + R_f \mathbf{h}_{t-1} + \mathbf{b}_f)$
candidate	$\mathbf{g}_t = \tanh(W_g \mathbf{x}_t + R_g \mathbf{h}_{t-1} + \mathbf{b}_g)$
output gate	$\mathbf{o}_t = \sigma(W_o \mathbf{x}_t + R_o \mathbf{h}_{t-1} + \mathbf{b}_o)$

cell state	$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \mathbf{g}_t$
hidden state	$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t)$

LSTMS: APPLICATIONS & SUCCESS IN NLP

- Language modeling (Mikolov et al., 2010; Sundermeyer et al., 2012)
- Parsing (Vinyals et al., 2015; Kiperwasser and Goldberg, 2016; Dryer et al., 2016)
- Machine translation (Bahdanau et al.,2015)
- Image captioning (Bernardi et al., 2016)
- Visual question answering (Antol et al., 2015)
- … and many other tasks!

6. TREE LSTM

SENTENCE REPRESENTATIONS WITH NNS

- **Bag of Words models**
  - sentence representations are **order-independent** functions of the word representations

- **Sequence models**
  - sentence representations are an **order-sensitive** function of a sequence of word representations (surface form)

- **Tree-structured models**
  - sentence representations are a function of the word representations, **sensitive to the syntactic structure** of the sentence

SECOND APPROACH: SENTENCE + SENTIMENT + SYNTAX

1. one-sentence review + “global” sentiment score
2. **tree structure (syntax)**
3. node-level sentiment scores

EXPLOITING TREE STRUCTURE

Instead of treating our input as a **sequence**, we can take an alternative approach:
assume a **tree structure** and use the principle of **compositionality**.

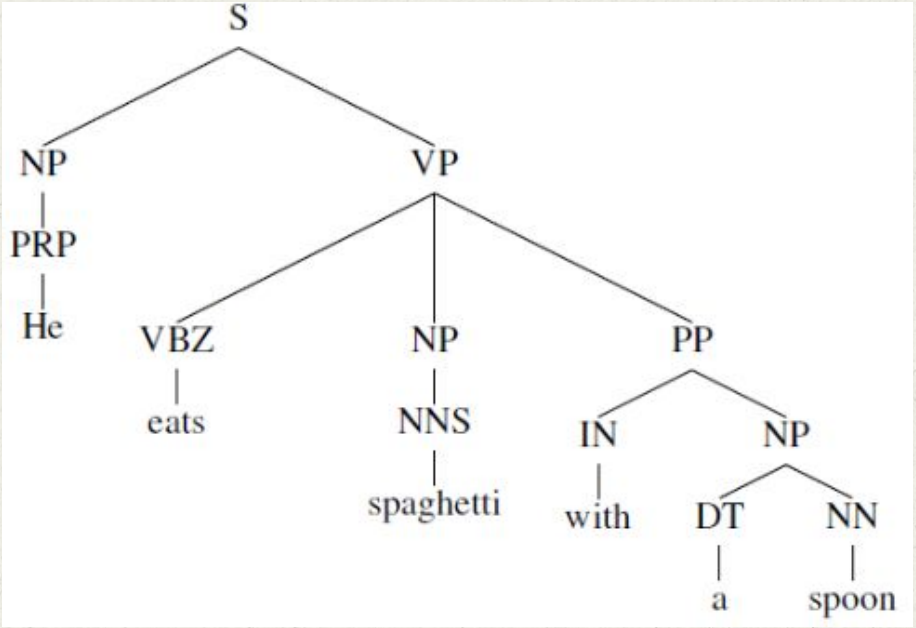
The meaning (vector) of a sentence is determined by:

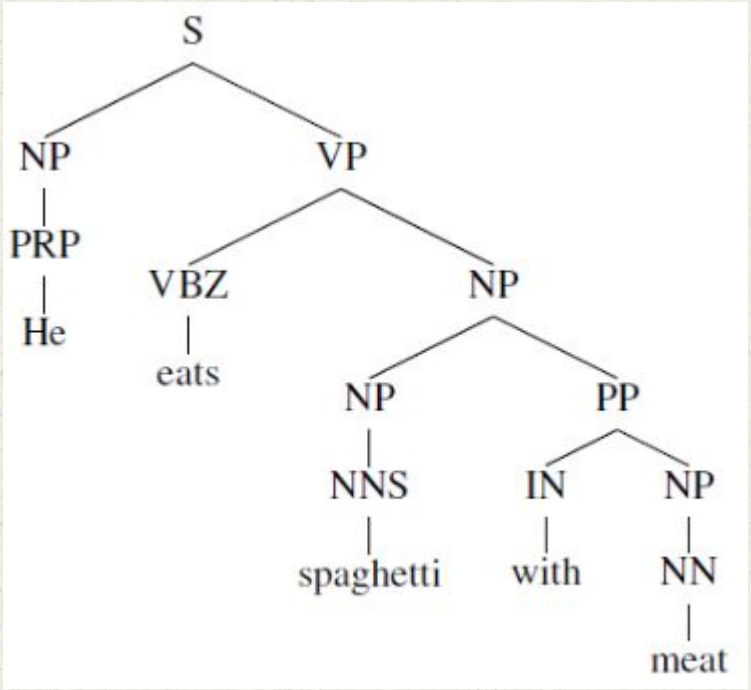
1. the meanings of its **words** and
2. the **rules** that combine them

Adapted from Stanford cs224n.

WHY WOULD IT BE USEFUL?

Helpful in **disambiguation**: similar “surface” / different structure

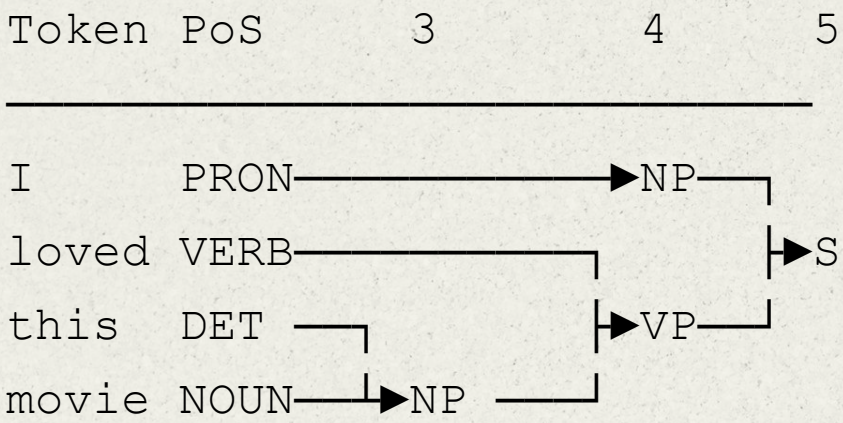




Credits: https://cs224d.stanford.edu/lectures/CS224d-Lecture10.pdf

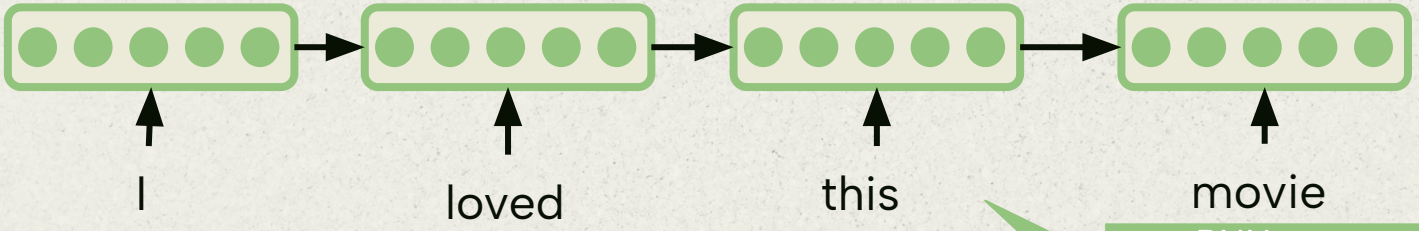
CONSTITUENCY PARSE

Can we obtain a **sentence vector** using the tree structure given by a parse?



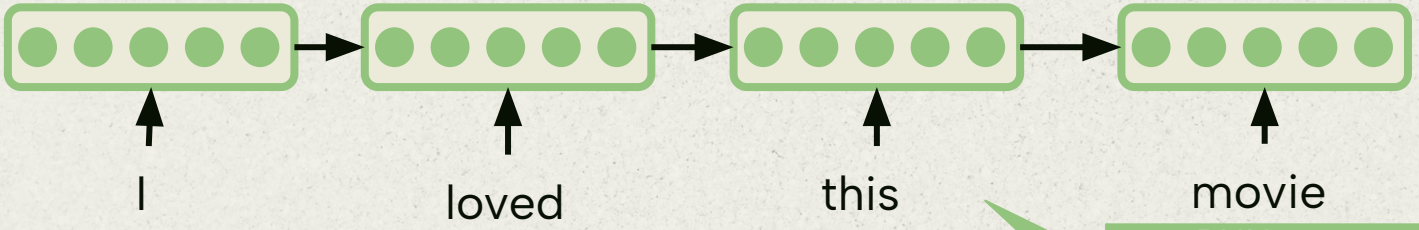
https://hanlp.hankcs.com/en/demos/con.html

RECURRENT VS TREE RECURSIVE NN



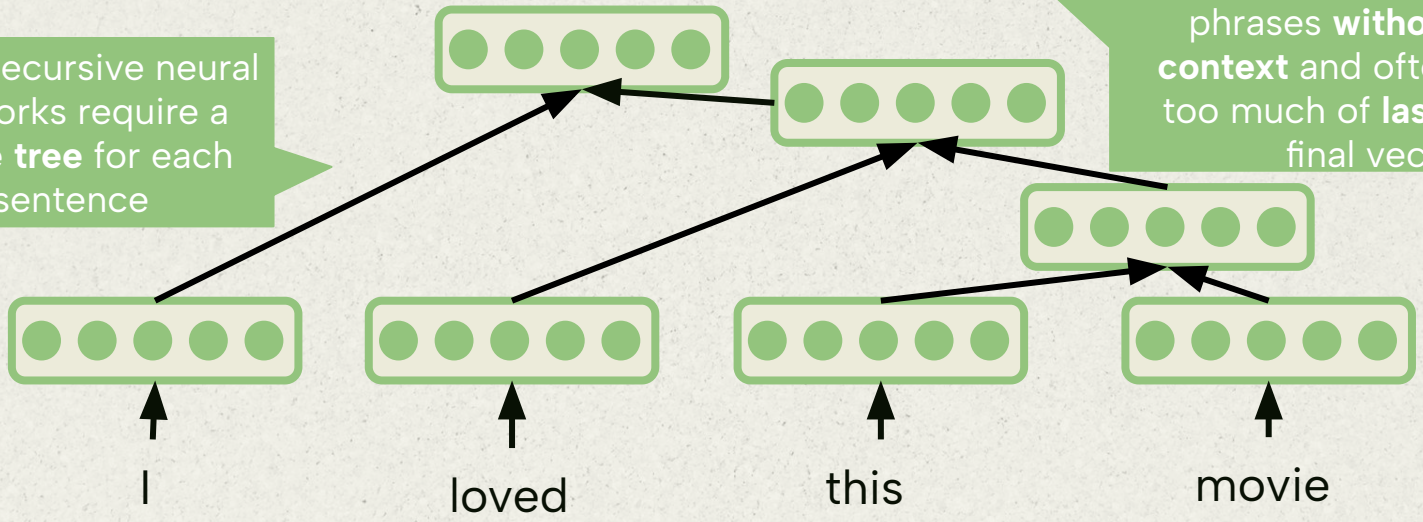
RNNs cannot capture phrases **without prefix context** and often capture too much of **last words** in final vector

RECURRENT VS TREE RECURSIVE NN



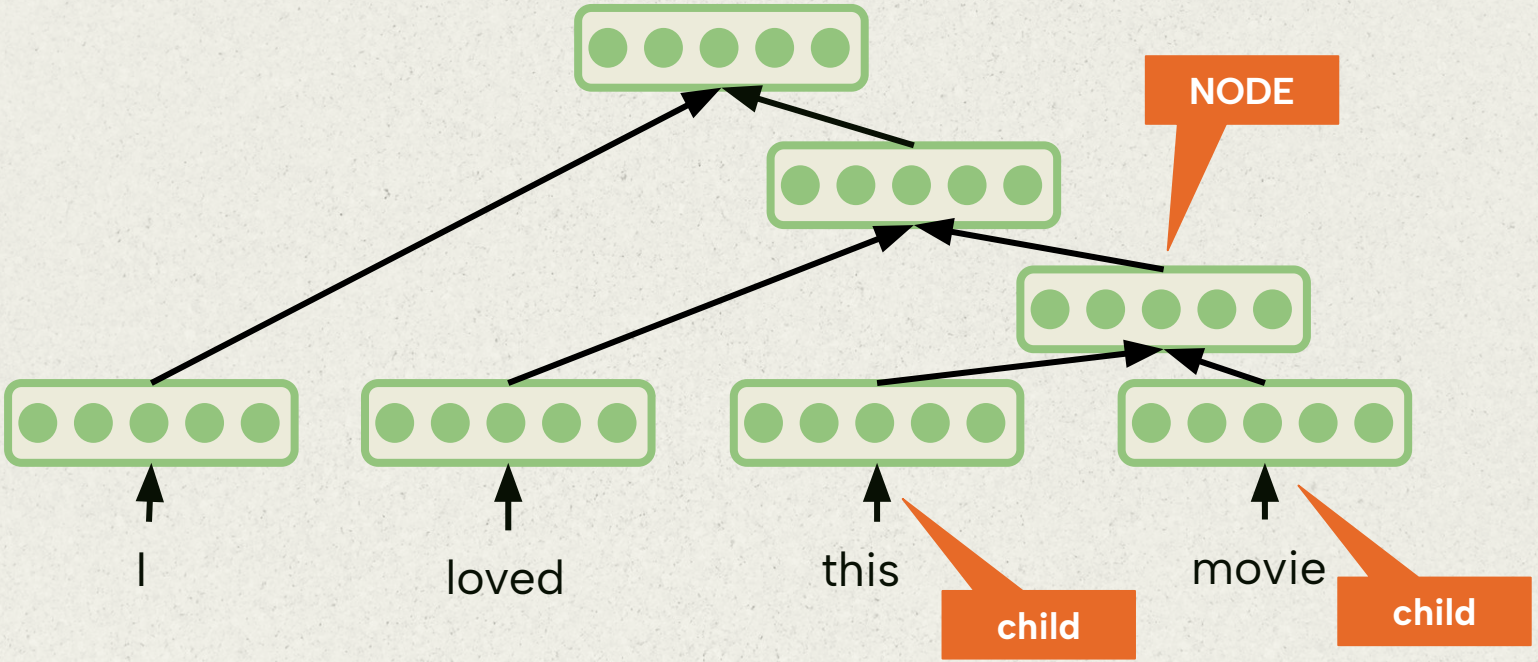
RNNs cannot capture phrases **without prefix context** and often capture too much of **last words** in final vector

Tree Recursive neural networks require a **parse tree** for each sentence

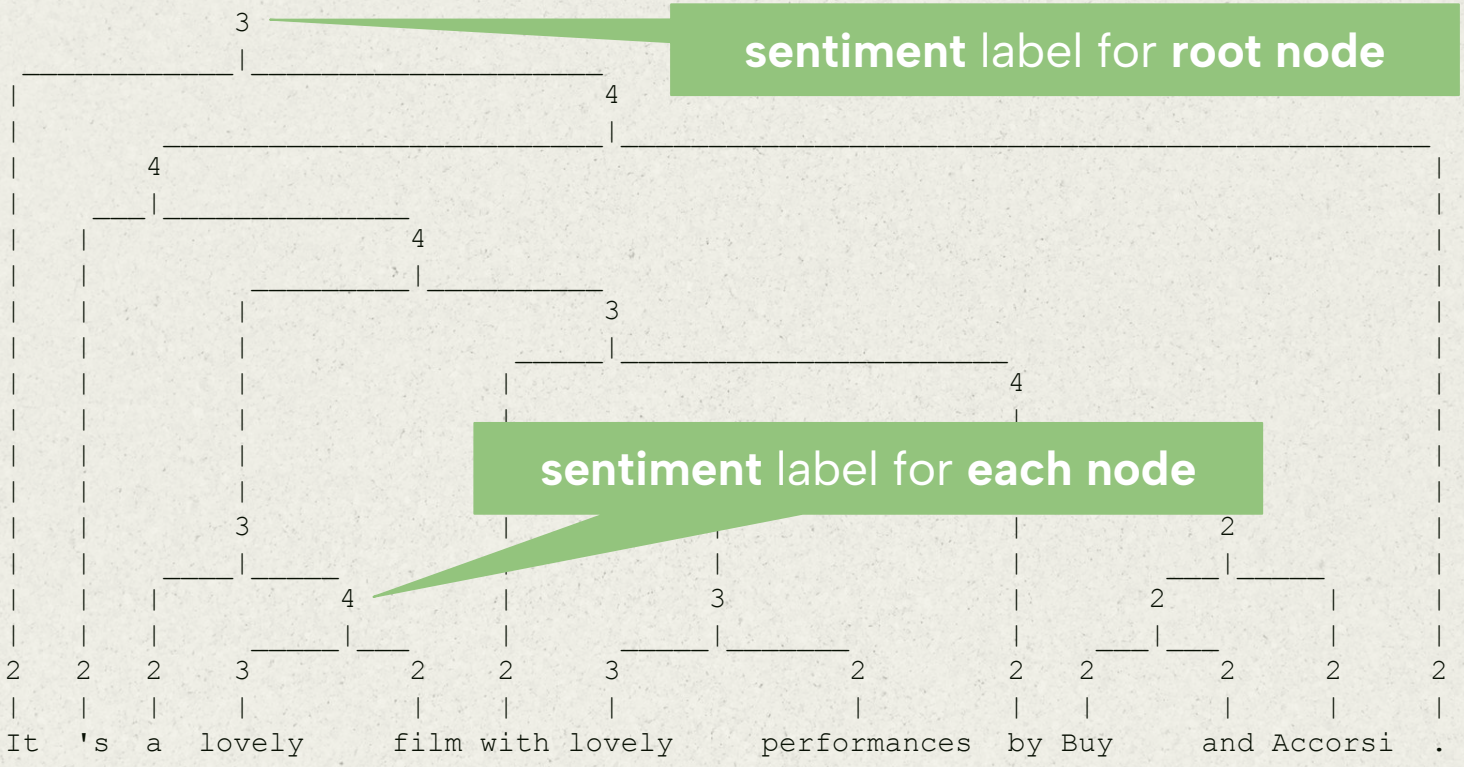


Adapted from Stanford cs224n.

TREE RECURSIVE NN



STANFORD SENTIMENT TREEBANK (SST)



TREE LSTMS: GENERALIZE LSTM TO TREE STRUCTURE

Use the idea of LSTM (gates, memory cell) but allow for multiple inputs (**node children**)

Proposed by 3 groups in the same summer:

- Kai Sheng Tai, Richard Socher, and Christopher D. Manning. ***Improved Semantic Representations From Tree-Structured Long Short-Term Memory Networks***. ACL 2015.
  - Child-Sum Tree LSTM
  - **N-ary Tree LSTM**
- Phong Le and Willem Zuidema.
  Compositional distributional semantics with long short term memory. *SEM 2015.
- Xiaodan Zhu, Parinaz Sobihani, and Hongyu Guo.
  Long short-term memory over recursive structures. ICML 2015

TREE LSTMS

- Child-Sum Tree LSTM

  sums over all children of a node; can be used for any N of children

- N-ary Tree LSTM

  different parameters for each child; better granularity (interactions between children) but maximum N of children per node has to be fixed

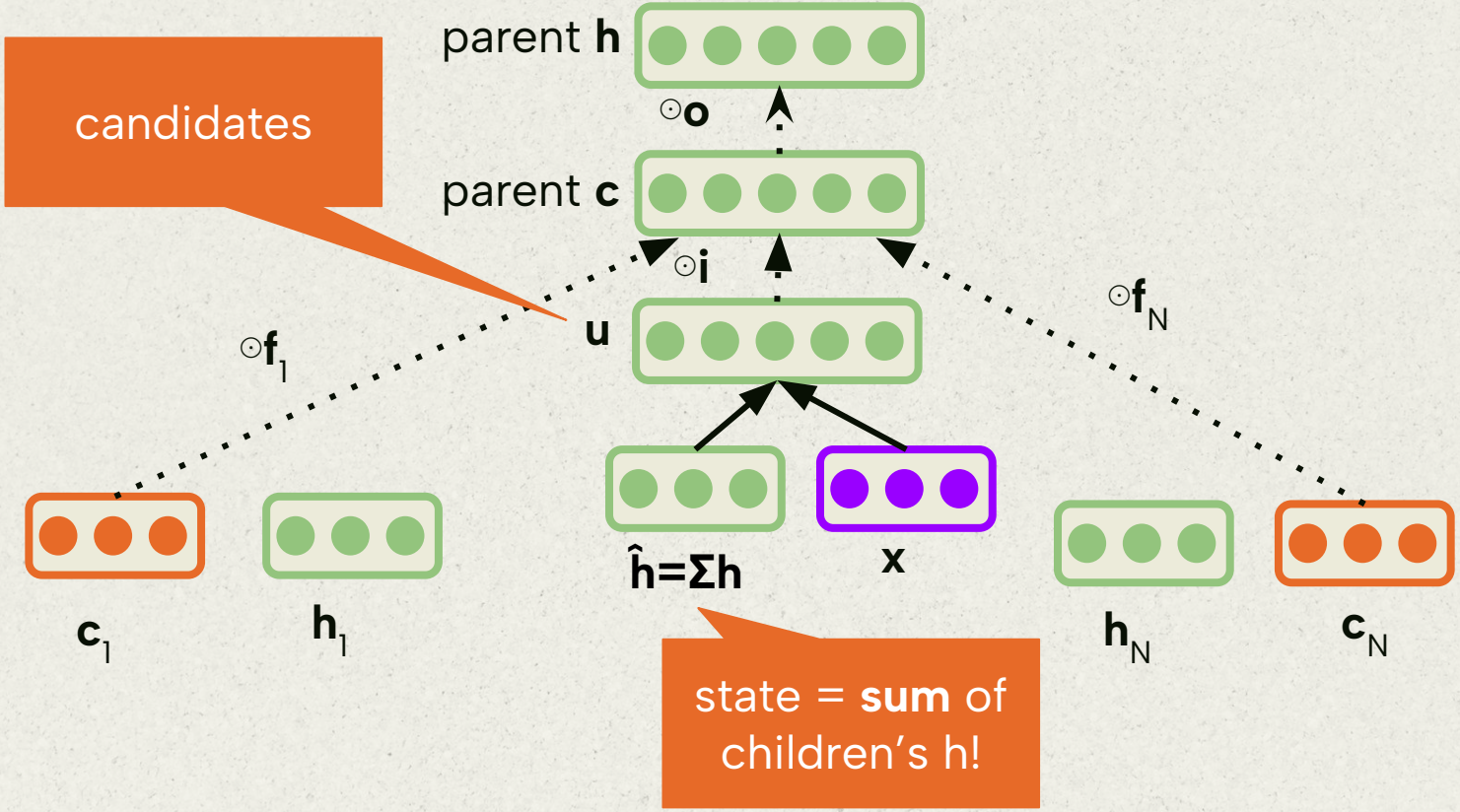
Credits: Daniel Perez https://www.slideshare.net/tuvistavie/tree-lstm

CHILD-SUM TREE LSTM

Children **outputs** and **memory cells** are **summed**

1. NO children order
2. works with variable number of children (sum!)
3. shares gates weights between children

CHILD-SUM TREE LSTM

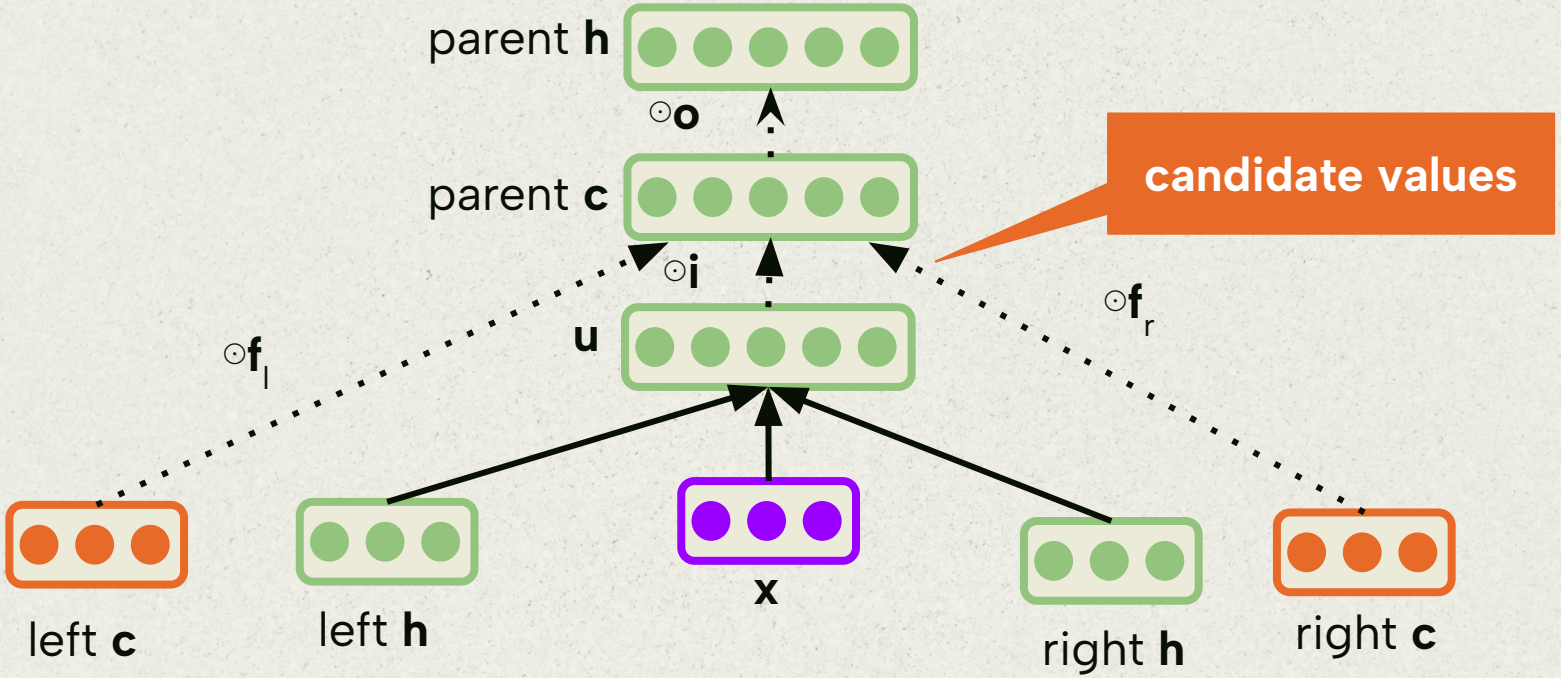


N-*ARY* TREE LSTM

Separate parameter matrices for each child *k*

1. each node must have at most N (e.g. **binary**) ordered children
2. fine-grained control on how information propagates
3. forget gate can be parametrized (N matrices, one per k) so that siblings affect each other

N-ARY TREE LSTM



N-ARY TREE LSTM

useful for encoding **constituency** trees

$$i_j = \sigma\left(W^{(i)}x_j + \sum_{\ell=1}^{N} U_\ell^{(i)} h_{j\ell} + b^{(i)}\right),$$

$$f_{jk} = \sigma\left(W^{(f)}x_j + \sum_{\ell=1}^{N} U_{k\ell}^{(f)} h_{j\ell} + b^{(f)}\right),$$

$$o_j = \sigma\left(W^{(o)}x_j + \sum_{\ell=1}^{N} U_\ell^{(o)} h_{j\ell} + b^{(o)}\right),$$

$$u_j = \tanh\left(W^{(u)}x_j + \sum_{\ell=1}^{N} U_\ell^{(u)} h_{j\ell} + b^{(u)}\right),$$

$$c_j = i_j \odot u_j + \sum_{\ell=1}^{N} f_{j\ell} \odot c_{j\ell},$$

$$h_j = o_j \odot \tanh(c_j),$$

LSTMs vs Tree-LSTMs

Standard LSTMs be considered as (a special case of) Tree-LSTMs

TREE-LSTM VARIANTS

- **Child-Sum Tree-LSTM**
  - sum over the hidden representations of all children of a node (**no children order**)
  - can be used for a **variable** number of children
  - **shares parameters** between children
  - suitable for dependency trees

- **N-ary Tree-LSTM**
  - discriminates between **children node positions** (weighted sum)
  - **fixed** maximum branching factor: can be used with N children at most
  - **different parameters** for each child
  - suitable for constituency trees

TRANSITION SEQUENCE REPRESENTATION

BUILDING A TREE WITH A TRANSITION SEQUENCE

We can describe a **binary tree** using a *shift-reduce* **transition sequence**

(I (loved (this movie)))
S S S S RRR

BUILDING A TREE WITH A TRANSITION SEQUENCE

We can describe a **binary tree** using a *shift-reduce* **transition sequence**

(I (loved (this movie)))
S S S S RRR

We start with a buffer (queue) and an empty stack:

stack = []
buffer = queue([I, loved, this, movie])

Iterate through the transition sequence:

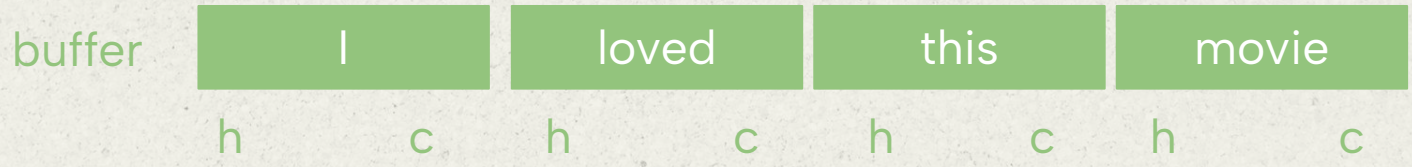
If SHIFT(S): take **first** word (*leftmost*) out of the **buffer**, push it to the **stack**

If REDUCE(R): **pop** top 2 words from **stack** + **reduce** them into a **new node (w/ tree LSTM)**

TRANSITION SEQUENCE EXAMPLE

(I (loved (this movie)))
 S S S S RRR

stack



TRANSITION SEQUENCE EXAMPLE

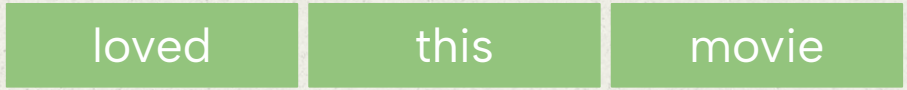
(I (loved (this movie)))

S S S S RRR



stack

buffer

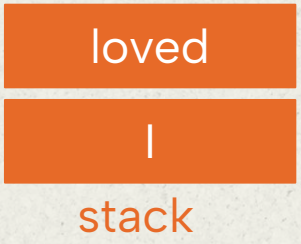


h c h c h c

TRANSITION SEQUENCE EXAMPLE

(I (loved (this movie)))

S S S S RRR

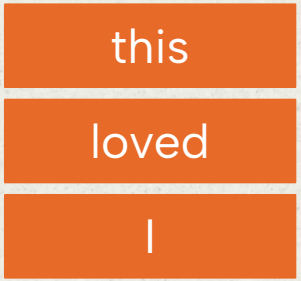




TRANSITION SEQUENCE EXAMPLE

(I (loved (this movie)))

S S S S RRR



stack

buffer



h c

TRANSITION SEQUENCE EXAMPLE

(I (loved (this movie)))

S S S S RRR

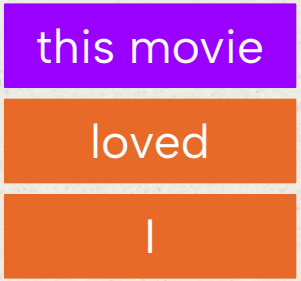


buffer

TRANSITION SEQUENCE EXAMPLE

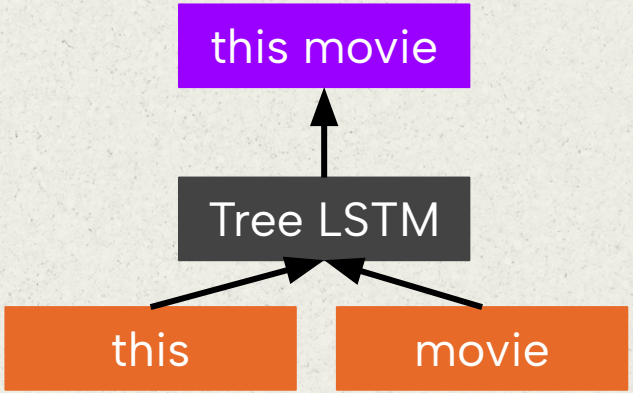
(I (loved (this movie)))

S S S S RRR



stack

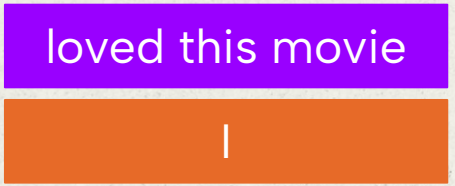
buffer



TRANSITION SEQUENCE EXAMPLE

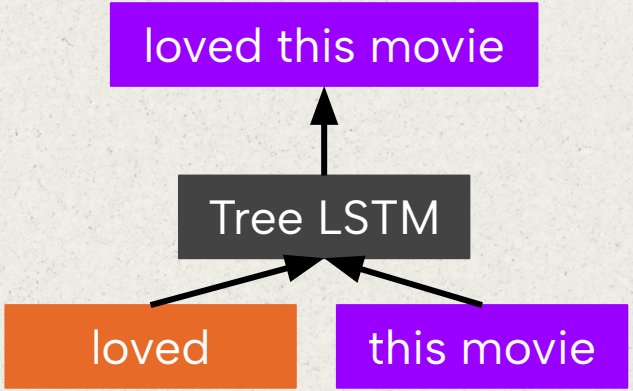
(I (loved (this movie)))

S S S S RRR



stack

buffer



TRANSITION SEQUENCE EXAMPLE

(I (loved (this movie)))

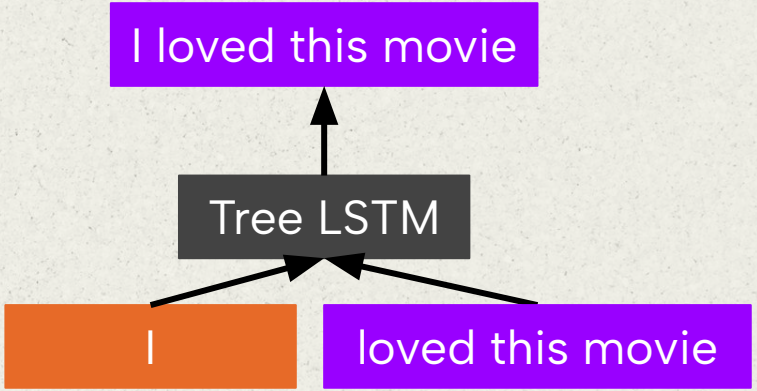
S S S S RRR

this is your **root node** for classification

I loved this movie

stack

buffer

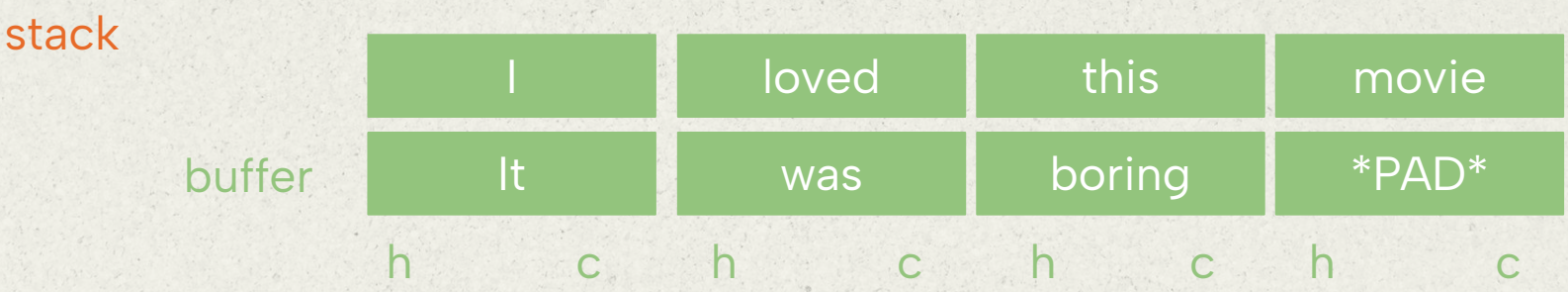


MINI-BATCH SGD

TRANSITION SEQUENCE EXAMPLE (MINI-BATCHED)

(I (loved (this movie)))
S S S S RRR

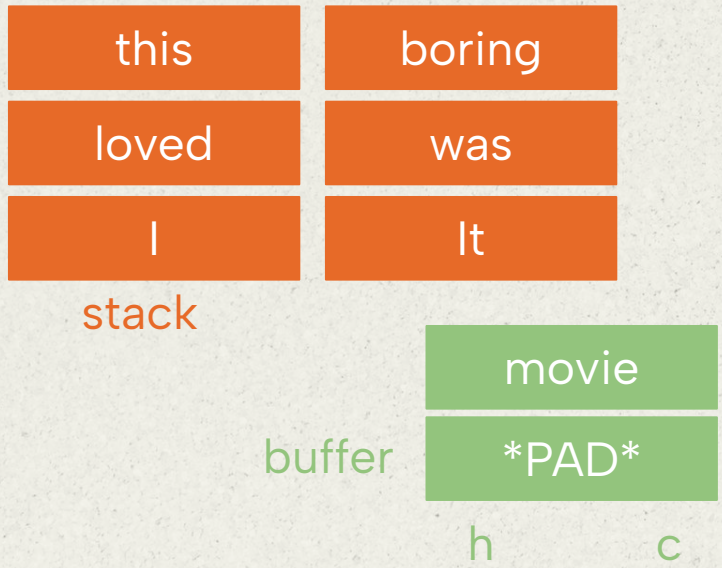
(It (was boring))
S S S R R



TRANSITION SEQUENCE EXAMPLE (MINI-BATCHED)

(I (loved (this movie)))
S S S S RRR

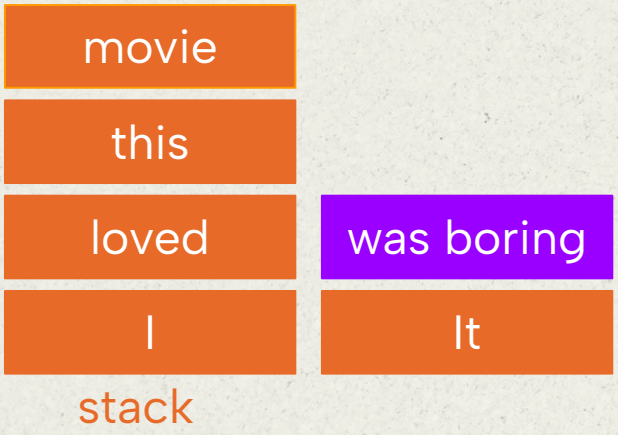
(It (was boring))
S S S R R

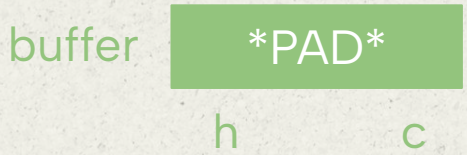


TRANSITION SEQUENCE EXAMPLE (MINI-BATCHED)

(I (loved (this movie))) (It (was boring))

S S S S RRR S S S R R





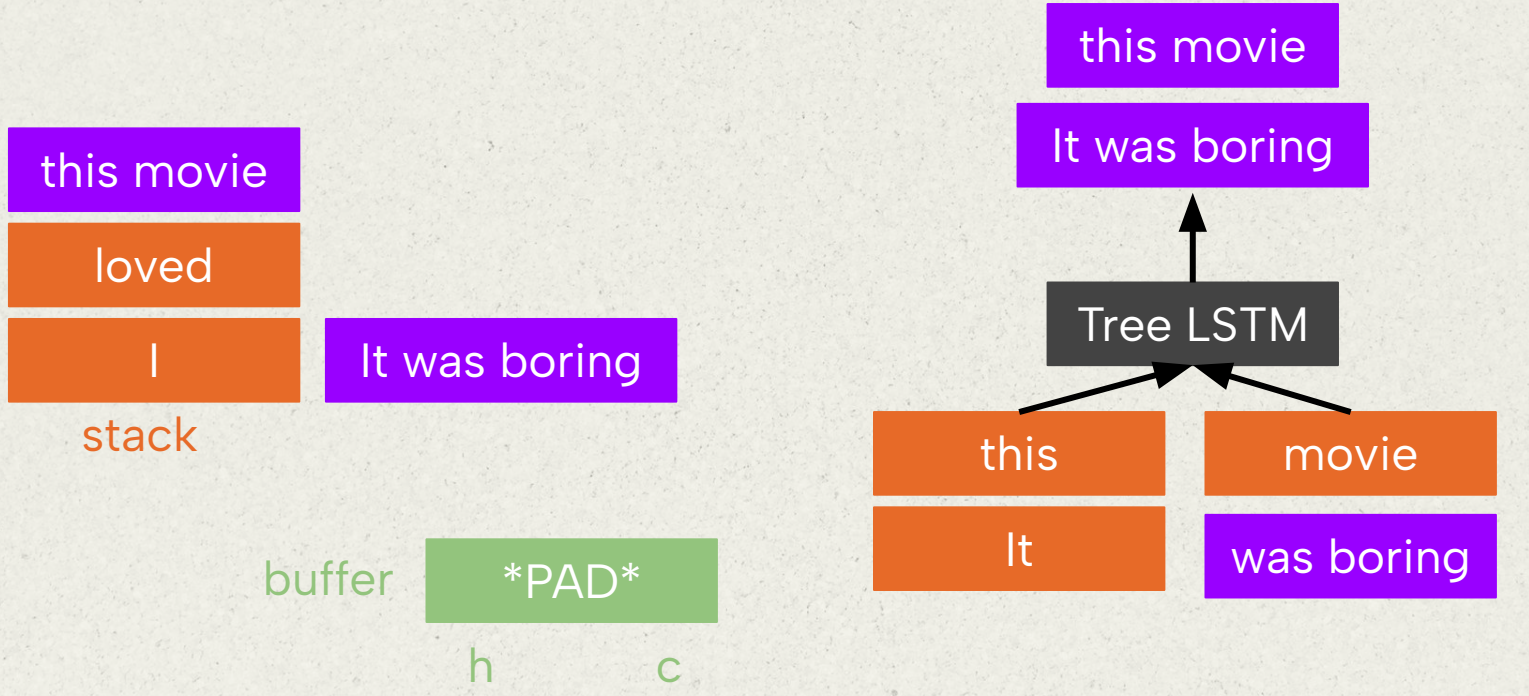
TRANSITION SEQUENCE EXAMPLE (MINI-BATCHED)

(I (loved (this movie)))

S S S S RRR

(It (was boring))

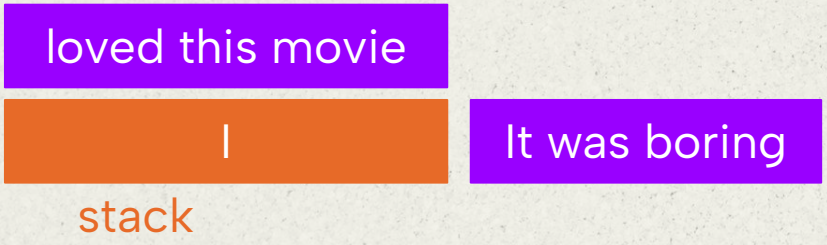
S S S R R

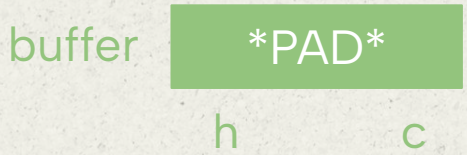


TRANSITION SEQUENCE EXAMPLE (MINI-BATCHED)

(I (loved (this movie)))
S S S S RRR

(It (was boring))
S S S R R





TRANSITION SEQUENCE EXAMPLE (MINI-BATCHED)

(I (loved (this movie)))
S S S S RRR

(It (was boring))
S S S R R

I loved this movie

It was boring

stack

buffer

PAD

h c

SUMMARY

RECAP

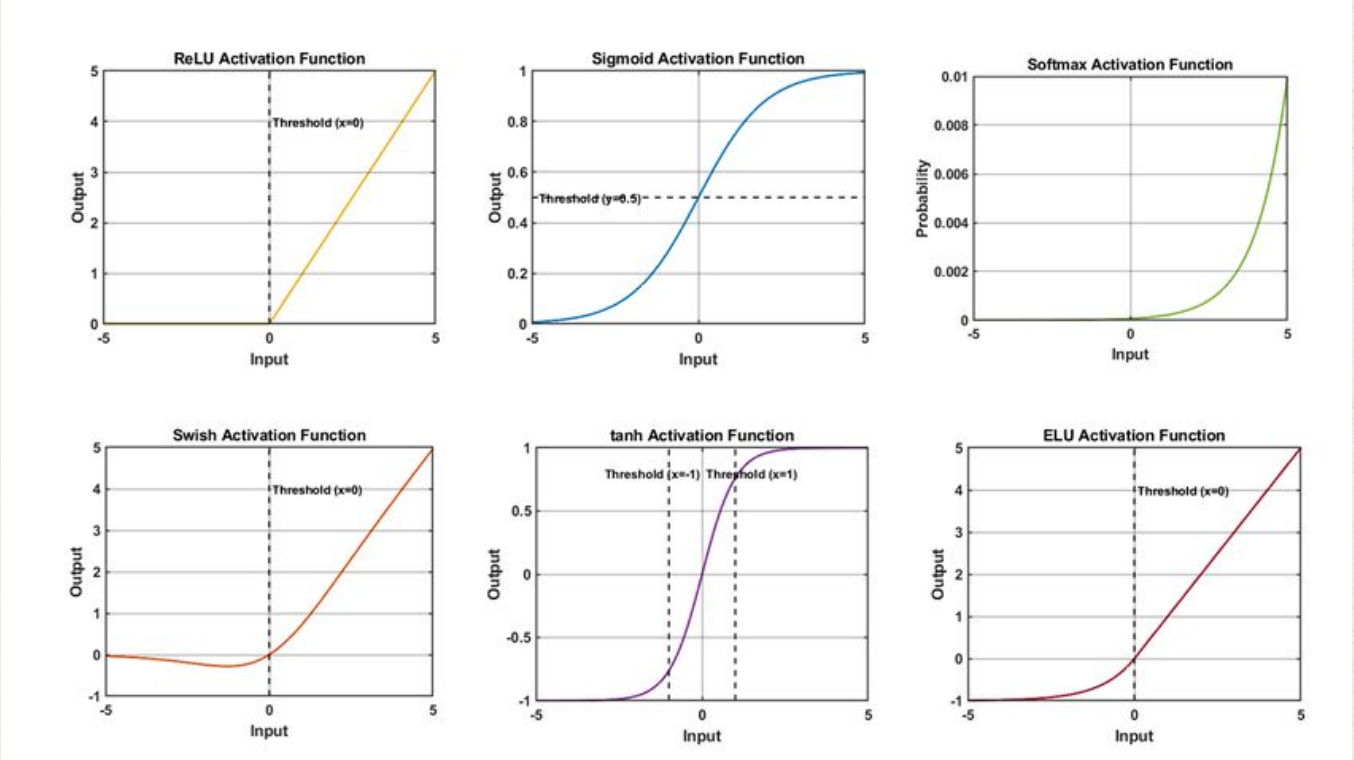
- Bag of Words models: BOW, CBOW, Deep CBOW
  - Can encode a sentence of arbitrary length, but loses word order
- Sequence models: RNN and LSTM
  - Sensitive to word order
  - RNN has vanishing gradient problem, LSTM deals with this
  - LSTM has input, forget and output gates that control information flow
- Tree-based models: Child-Sum & N-ary Tree LSTM
  - Generalize LSTM to tree structures
  - Exploit compositionality, but require a parse tree

EXTRA

INPUT

In a TreeLSTM over a constituency tree (ours!), the leaf nodes take the corresponding word vectors as input

RECAP: ACTIVATION FUNCTIONS



CHILD-SUM TREE LSTM

useful for encoding **dependency** trees

$$\tilde{h}_j = \sum_{k \in C(j)} h_k,$$

$$i_j = \sigma\left(W^{(i)} x_j + U^{(i)} \tilde{h}_j + b^{(i)}\right),$$

$$f_{jk} = \sigma\left(W^{(f)} x_j + U^{(f)} h_k + b^{(f)}\right),$$

$$o_j = \sigma\left(W^{(o)} x_j + U^{(o)} \tilde{h}_j + b^{(o)}\right),$$

$$u_j = \tanh\left(W^{(u)} x_j + U^{(u)} \tilde{h}_j + b^{(u)}\right),$$

$$c_j = i_j \odot u_j + \sum_{k \in C(j)} f_{jk} \odot c_k,$$

$$h_j = o_j \odot \tanh(c_j),$$